



TECHNICAL ARCHITECTURE & SYSTEM DOCUMENTATION

XaminityIQ

Online Examination & Proctoring Platform

A full-stack academic examination system covering institution setup, exam authoring, live invigilation, and result publishing — documented from product intent down to request/response contracts.

DOCUMENT

Technical Architecture & System Documentation

VERSION

1.0

STATUS

Final

DATE

31 August 2026

AUTHOR / DEVELOPER

Sanjai Kannan G

CLASSIFICATION

Internal / Portfolio Reference

PRIMARY STACK

React 19 · NestJS 11 · MongoDB · LiveKit

REPOSITORIES

XaminityIQ-Server · XaminityIQ-Client

Document Information

FIELD	DETAILS
Project	XaminiyIQ — Online Examination & Proctoring Platform
Document	Technical Architecture & System Documentation
Version	1.0
Status	Final
Date	31 August 2026
Author	Sanjai Kannan G
Primary Technology	React 19 / TypeScript (frontend), NestJS 11 / TypeScript (backend), MongoDB Atlas (database)
Repositories	<code>XaminiyIQ-Server</code> (backend, branch <code>feat/exam-module</code>), <code>XaminiyIQ-Client</code> (frontend)

Revision History

VERSION	DATE	CHANGES	AUTHOR
1.0	31 Aug 2026	Initial technical architecture and system documentation covering the full implemented feature set as of the <code>feat/exam-module</code> branch.	Sanjai Kannan G

Purpose of This Document

This document describes the XaminiyIQ platform's architecture, modules, data model, API surface, security model, and operational characteristics as they exist in the current codebase. It is intended to let a new engineer understand the system well enough to contribute to it, and to serve as a reference during architecture discussions, code review, and further development.

SCOPE OF ACCURACY

Every technical claim in this document is derived directly from the source code of the two repositories listed above (controllers, services, Mongoose schemas, DTOs, guards, configuration files) or from git history where noted. Where a fact could not be confirmed from the implementation — for example, the exact production hosting provider for the backend, which is referenced only in a code comment and not in a deployment manifest — it is explicitly marked **To be verified** rather than presented as confirmed.

Related Documents

The two repositories each carry a `README.md` with setup instructions. At the time of writing, both READMEs contain environment-variable names that do not match the variables actually read by the application code (see Section 23, Environment Configuration) — this document supersedes them for architectural and configuration reference purposes.

Table of Contents

Entries are clickable links to each chapter in this PDF.

1. Project Overview

2. Project Scope

3. System Architecture

4. Technology Stack

5. Application Modules

6. User Roles & Responsibilities

7. Application Workflows

8. Frontend Architecture

9. Backend Architecture

10. API Documentation

11. API Access-Control Matrix

12. Database Architecture

13. Entity Relationship Diagrams

14. Data Flow Diagrams

15. Sequence Diagrams

16. Examination System

17. Proctoring Architecture

18. Authentication & Authorization

19. Security Architecture

20. External Services & Integrations

21. File & Storage Architecture

22. Deployment Architecture

23. Environment Configuration

24. Logging & Monitoring

25. Technical Challenges & Solutions

26. Architecture & Design Decisions

27. Testing Strategy

28. Performance Considerations

29. Error Handling & Validation

30. Feature Traceability Matrix

31. Future Improvements

32. Conclusion

33. Glossary

34. Appendix

1. Project Overview

XaminityIQ is a full-stack academic examination platform that lets an institution manage its academic structure, run both self-paced and live-invigilated online exams, and monitor students in real time during proctored sessions — as a single connected system rather than a set of disconnected tools.

1.1 Business Perspective

Institutions running online exams typically face three separate problems: keeping student/faculty/academic records organized, authoring and scheduling exams without manual coordination, and — for high-stakes exams — supervising students remotely without a physical exam hall. XaminityIQ addresses all three within one application, using the same underlying data (a student's batch/course/department/section/semester, an exam's question bank, a faculty member's invigilation assignment) across every step from onboarding to result publication.

Target users

- **Educational institutions** (colleges/departments) that need to run scheduled exams across multiple batches, courses, and sections.
- **Administrative staff (Super Admin)** who set up the academic hierarchy, onboard students and faculty, and create/publish exams.
- **Faculty** who manage their subjects and invigilate the proctored exams they are assigned to.
- **Students** who take exams — either self-paced or under live camera/screen supervision.

Problems the platform solves

- No unified place to model batches, courses, departments, sections, and subjects together — the platform enforces this hierarchy at the data-model level, so an exam can only target combinations that actually exist.
- Manually assigning invigilators and splitting students into exam rooms — the platform automates room formation, partitioning matched students into groups and round-robin assigning active faculty as invigilators.
- No real-time visibility into a remote exam session — the platform streams each admitted student's camera and screen to their assigned faculty member over WebRTC, with faculty-driven admission, mic control, and removal.
- Slow, disconnected evaluation of subjective answers — Written and Typing answers flow into a faculty-scoped evaluation queue tied directly to the same exam and attempt records used everywhere else in the system.

1.2 Technical Perspective

XaminityIQ is a two-repository system: a NestJS/TypeScript REST API backed by MongoDB (Mongoose ODM), and a React 19/TypeScript single-page application. The two communicate exclusively over a versioned-by-convention REST API secured with short-lived JWT access tokens and an httpOnly-cookie refresh token. Real-time proctoring (camera, screen-share, and in-exam chat) is implemented over LiveKit, a WebRTC SFU, with the

NestJS backend issuing scoped, per-room join tokens rather than proxying media itself. File assets — profile photos, written handwritten-answer page photographs, and exam-session recording chunks — are stored in Cloudinary via signed, time-limited upload credentials generated by the backend, so the client never holds long-lived storage credentials.

The system is organized around three first-class roles (Super Admin, Faculty, Student), each with its own route tree, dashboard, and API surface, enforced identically at both the frontend (route guards) and backend (JWT authentication + role guards on every protected controller method) layers.

1.3 Major Capabilities

CAPABILITY	SUMMARY
Academic structure management	Batches, courses, departments, sections, and subjects, with the mapping relationships between them (which courses belong to a batch, which departments belong to a batch-course, etc.).
User management	Student and faculty onboarding (individual and bulk CSV), self-serve profile completion, edit/deactivate/activity history.
Exam authoring	AUTO (self-paced) and PROCTORING (invigilated) exam modes; MCQ, MSQ, Written, and Typing question types; exam sections; bulk question CSV upload; per-exam security settings.
Exam room formation	Automatic partitioning of matched students into rooms of up to 10, with round-robin faculty invigilator assignment and pooling across window-sibling exams that share an identical proctoring schedule.
Live proctoring	Waiting-room admission/rejection, live camera and screen-share monitoring, faculty-controlled mic muting, in-session chat, disconnect/reconnect handling with a configurable grace period, and integrity monitoring (tab-switch/fullscreen-exit detection with an auto-submit threshold).
Evaluation & results	A faculty-scoped evaluation queue for subjective (Written/Typing) answers, marks and remarks entry, and admin-controlled result publication.
Dashboards	Role-specific dashboards for Super Admin, Faculty, and Student.

1.4 High-Level Workflow

At the highest level, the platform's lifecycle runs: an admin builds the academic hierarchy and onboards users → an admin (or the exam's owning faculty context) authors and publishes an exam against that hierarchy → for a proctored exam, rooms are formed and invigilators assigned → students take the exam (with or without live supervision, depending on mode) → subjective answers are evaluated → results are published. Section 7 (Application Workflows) documents each of these paths as a step-by-step diagram.

1.5 Key Technical Characteristics

- **Layered backend** — every backend module follows the same `api → service → repository → schema` layering, so controllers never talk to Mongoose models directly (see Section 9).
- **Schema-derived authorization** — a student's eligibility for an exam is computed from their own academic-detail record against the exam's target hierarchy at request time, not cached or duplicated.
- **Time-driven state machines** — exam status (`DRAFT → PUBLISHED → ONGOING → COMPLETED → RESULTS_PUBLISHED`) and room-assignment status both advance automatically based on real time via a scheduled sweep, rather than relying on a user action to transition state (see Section 25 for a real defect this exposed and how it was fixed).
- **No server-side media relay** — proctoring video/audio/screen-share travels directly between the student's browser, the LiveKit SFU, and the faculty's browser; the NestJS backend only issues signed tokens and persists session metadata.

2. Project Scope

This section distinguishes what is actually implemented today from what is deliberately excluded and what could reasonably be added later, so the rest of this document is never read as describing planned functionality as if it already existed.

2.1 In Scope (Implemented)

- Academic hierarchy management: batches, courses, batch-course mapping, departments, batch-department mapping, sections (with capacity), subjects (with subject codes, tied to a department and semester).
- Student management: creation (individual and bulk CSV), edit, soft delete/deactivation, activity history, self-serve profile completion (address, emergency contact, education history, parent/guardian details, profile photo) after first login.
- Faculty management: creation (individual), edit, soft delete/deactivation, activity history, self-serve profile completion (address, emergency contact, education history, work experience, profile photo), subject ownership.
- Exam authoring: AUTO and PROCTORING modes; MCQ, MSQ, Written, and Typing question types; named exam sections; bulk question CSV upload; shuffle-questions/shuffle-options, minimum-time-per-question, minimum-time-per-exam, and marks-integrity validation; multi-section and multi-semester exam targeting; Draft → Published → Ongoing → Completed → Results-Published lifecycle, time-driven where noted.
- Exam room formation for PROCTORING exams: automatic student partitioning (≤ 10 per room), round-robin faculty invigilator assignment, and pooling of leftover students across window-sibling exams sharing an identical schedule.
- Live proctoring: pre-flight environment check (camera, microphone, screen-share, single/multi-monitor detection, fullscreen, network probe), waiting-room join, faculty admit/reject, live camera and screen-share monitoring over LiveKit, faculty-controlled mic mute, in-session chat (broadcast and 1:1), disconnect/reconnect handling with a configurable grace period, and integrity monitoring (tab-switch and fullscreen-exit detection with a configurable auto-submit violation threshold).
- Student exam-taking: MCQ/MSQ answer capture, Typing rich-text answers, Written answers captured via a phone-scanned QR code and photograph upload, per-question and per-exam minimum-time enforcement, countdown timer, manual and automatic (timer-expiry / integrity-violation) submission.
- Evaluation: faculty-scoped evaluation queue for Written and Typing answers, marks and remarks entry, admin per-attempt answer review (all question types), recording playback, admin-controlled result publication.
- Authentication & authorization: JWT access/refresh token authentication, role-based access control for three roles, protected routes on both frontend and backend, forgot-password / reset-password flow.
- Dashboards: role-specific overview dashboards for Super Admin, Faculty, and Student, including a public marketing/landing page.

2.2 Out of Scope (Not Implemented)

The following are intentionally absent from the current implementation — they are not partially built, stubbed, or planned for the immediate roadmap unless noted in Section 2.3:

- Any payment or billing functionality.
- Email or SMS notification delivery — the codebase contains no mail/SMS provider integration; all user-facing feedback is synchronous, in-app (toast messages) or QR-code based.
- Automated grading of subjective (Written/Typing) answers — all Written and Typing scoring is manual, via the faculty evaluation queue.
- Plagiarism or answer-similarity detection.
- A native mobile application — the "mobile" written-answer capture flow is a responsive web page opened by scanning a QR code in a phone browser, not an installed app.
- Multi-language / internationalization support.
- Rate limiting on any API endpoint (confirmed absent — no `@nestjs/throttler` or equivalent is present in the backend dependencies).
- Automated CI/CD pipelines — no GitHub Actions workflows or equivalent exist in either repository at the time of writing.
- Project-specific automated test coverage — Jest and Supertest are installed and configured in the backend, but the only test files present are the default NestJS-generated smoke tests (`app.controller.spec.ts` , `app.e2e-spec.ts`); the frontend has no test tooling installed at all. See Section 27 (Testing Strategy).
- Third-party single sign-on (Google/Microsoft OAuth) — authentication is exclusively email/password against the platform's own user store.

2.3 Future Scope (Potential Enhancements)

These are reasonable extensions given the current architecture, not commitments or in-progress work — see Section 31 for the fuller discussion.

- Email/SMS notifications for exam scheduling, result publication, and account actions.
- Automated or AI-assisted first-pass scoring for Written/Typing answers, with faculty override.
- Rate limiting and stricter abuse protection on public/authentication endpoints.
- Automated test coverage for the service layer and critical API flows.
- CI/CD pipelines for both repositories.
- An audit-log/activity-trail view for admin-facing security review, building on the activity endpoints that already exist for students and faculty.

3. System Architecture

XaminityIQ is a two-repository client/server system: a React single-page application and a NestJS REST API, backed by MongoDB and three external services. There is no API gateway product in front of the backend — the term below refers to the backend's own guard/pipe pipeline, which plays that role.

3.1 High-Level Architecture

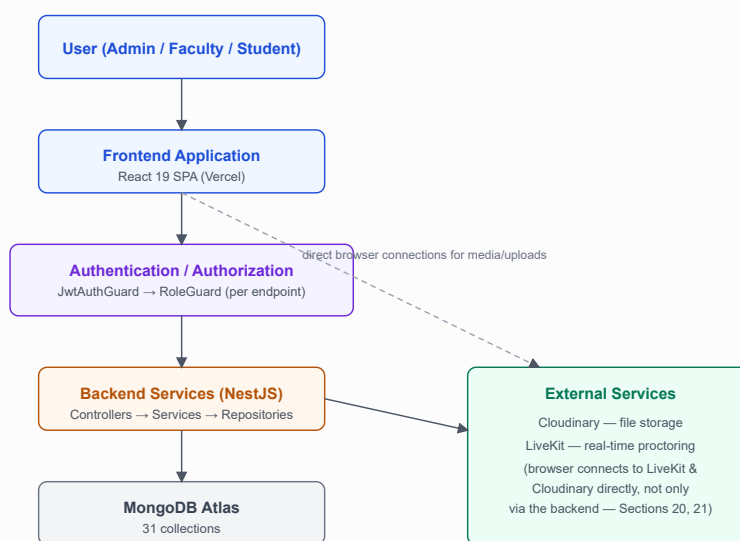


Figure 3.1 — High-level architecture. Unlike a typical AWS-centric stack, there is no managed API gateway, message queue, or serverless compute layer — the NestJS process is a single monolithic API server, and "authorization" is a guard pipeline inside that same process, not a separate service.

3.2 Layer Responsibilities

LAYER	RESPONSIBILITY
Client / Application Layer	React SPA — rendering, client-side routing/guards, form validation, RTK Query caching, direct LiveKit/Cloudinary connections for real-time media and file upload.
Authentication / Authorization Layer	<code>JwtAuthGuard</code> + <code>RoleGuard</code> , applied per-endpoint inside the NestJS request pipeline (Section 9.2) — not a separate deployed service.
API Layer	NestJS controllers — one per endpoint file, responsible only for request/response shaping and delegating to a service (Section 10).
Business Logic Layer	NestJS services — all domain rules (Section 9.4, Section 5).

Database / Storage Layer	MongoDB Atlas (structured data, Section 12) + Cloudinary (files, Section 21).
External Services	LiveKit (real-time proctoring media, Section 20); no other third-party service is integrated (Section 20.4).

3.3 Repository Topology

Two independently deployed, independently versioned repositories communicate exclusively over HTTPS REST — there is no shared code, monorepo tooling, or internal RPC between them. The frontend holds no server secrets; every credential (JWT signing keys, Cloudinary secret, LiveKit secret) lives only in the backend's environment configuration (Section 23).

4. Technology Stack

Every technology listed below is a confirmed dependency in `package.json` of the respective repository — nothing in this section is inferred or assumed.

4.1 Frontend

TECHNOLOGY	VERSION	ROLE
React	<code>^19.0.0</code>	UI library — the entire client is a single-page application.
TypeScript	<code>~5.7.2</code>	Static typing across the whole client codebase.
Vite	<code>^6.2.0</code>	Dev server and production bundler.
Tailwind CSS	<code>^4.1.10</code>	Utility-first styling, configured CSS-first via <code>@theme</code> in <code>src/global.css</code> (no separate <code>tailwind.config.js</code> — Tailwind v4 style).
Redux Toolkit	<code>^2.8.2</code>	Global store; in practice used almost exclusively to host RTK Query's cache — see Section 8.3.
RTK Query	(part of <code>@reduxjs/toolkit</code>)	Data fetching/caching layer, with a custom Axios-based <code>baseQuery</code> instead of the default <code>fetchBaseQuery</code> .
React Router	<code>^7.7.1</code>	Client-side routing, nested layouts, role-scoped route guards.
Axios	<code>^1.11.0</code>	HTTP client, wrapped with request/response interceptors for token attachment and silent refresh.
Formik	<code>^2.4.6</code>	Form state management — used on every data-entry form in the application.
Yup	<code>^1.6.1</code>	Schema-based form validation, paired 1:1 with Formik.
React Select / react-select-async-paginate	<code>^5.10.2 / ^0.7.11</code>	Searchable and paginated dropdowns for hierarchy pickers (batch/course/department/section/subject).
Lucide React	<code>^0.526.0</code>	Icon set — the only icon library in the project.
Tiptap (react/starter-kit/pm)	<code>^2.11.5</code>	Rich-text editor for the Typing question-answer type.
LiveKit Client	<code>^2.21.0</code>	WebRTC client SDK for live proctoring (camera, microphone, screen-share, data-channel chat).
QRCode	<code>^1.5.4</code>	Client-side QR code generation for the written-answer mobile capture flow.
Papa Parse	<code>^5.4.1</code>	CSV parsing for bulk student and bulk question upload.

DOMPurify	^3.2.4	Sanitizes Tiptap-authored HTML before it is rendered anywhere (Typing answers), mitigating stored XSS.
ECharts	^6.0.0	Charting library, wrapped by a thin lifecycle component (<code>common/ui/Chart.tsx</code>) for dashboard visualizations.
React Hot Toast	^2.5.2	Toast notifications — the app's sole success/error-feedback mechanism (no notification service).

4.2 Backend

TECHNOLOGY	VERSION	ROLE
NestJS (core/common/platform-express)	^11.x	Backend application framework — module/controller/service/DTO architecture, dependency injection.
TypeScript	^5.x	Static typing across the backend codebase.
MongoDB	Atlas (cloud)	Primary datastore — a managed, SRV-connected Atlas cluster (<code>mongodb+srv://</code>).
Mongoose (@nestjs/mongoose)	—	ODM — every persisted entity is a Mongoose schema class decorated with NestJS's <code>@Schema</code> / <code>@Prop</code> .
class-validator / class-transformer	—	DTO validation and payload transformation, enforced globally via a <code>ValidationPipe</code> (whitelist, forbid-non-whitelisted, auto-transform).
Passport / passport-jwt	—	Underlying strategy consumed by the custom <code>JwtAuthGuard</code> .
bcrypt	—	Password hashing.
jsonwebtoken (via @nestjs/jwt)	—	Access-token, refresh-token, password-reset-token, and QR-upload-token signing/verification — several distinct signing secrets, see Section 22.
cookie-parser	—	Reads the httpOnly refresh-token cookie on incoming requests.
Cloudinary SDK	—	Signed upload credential generation for profile photos, written-answer page photographs, and exam-recording chunks.
LiveKit Server SDK	—	Access-token issuance (<code>AccessToken</code>), forced participant removal, and server-side mic mute (<code>RoomServiceClient</code>) for proctoring.
@nestjs/schedule	—	Powers the exam-lifecycle sweep and the LiveKit disconnect sweep (<code>@Interval</code> -based, not cron-expression based).
ulid	—	Sortable unique identifiers used where noted in room/session naming.

NOTE ON EXACT BACKEND VERSIONS

Package-level exact version numbers for the backend were not independently re-verified line-by-line for this revision; NestJS 11 and its ecosystem packages are confirmed via the framework's own API surface used throughout the codebase (guards, pipes, decorators consistent with the v11/v10 API). Treat individual backend patch versions as **to be verified** against `package.json` if exact pinning matters for a dependency-upgrade decision.

4.3 Authentication & Authorization

MECHANISM	DETAIL
Authentication	JWT access token (short-lived) + JWT refresh token (long-lived, stored as an httpOnly cookie, never exposed to client-side JavaScript).
Password storage	bcrypt-hashed; plaintext passwords are never persisted.
Password reset	A separate, single-purpose JWT (<code>PASSWORD_RESET_JWT_SECRET_KEY</code>), short-lived, emailed conceptually but delivered via the reset-password URL/token flow (no email provider is integrated — see Section 2.2).
Authorization	Role-based access control (RBAC) for exactly three roles — <code>ADMIN</code> , <code>FACULTY</code> , <code>STUDENT</code> — enforced per-endpoint via a custom <code>RoleGuard</code> reading a <code>@Roles()</code> decorator, and per-route on the frontend via a <code>RoleGuard</code> component plus layout-level auth HOCs.
QR-token authorization	A distinct, narrowly-scoped JWT (<code>QR_TOKEN_JWT_SECRET_KEY</code>) authorizes the unauthenticated mobile written-answer upload flow — the token itself is the credential, cross-checked against a stored hash and expiry on the answer record.

TO BE VERIFIED

No AWS Cognito, AWS Verified Permissions, Cedar, or any third-party identity provider is used anywhere in this codebase — authentication is fully self-hosted (JWT + bcrypt against the platform's own `User` / `Admin` / `Faculty` / `Student` collections). This is stated as a confirmed negative, not an assumption.

4.4 Cloud & External Services

XaminityIQ does not use AWS or any other general-purpose cloud provider's managed services. The three external services actually integrated are documented in full in Section 20 (External Services & Integrations):

SERVICE	CATEGORY	USED FOR
MongoDB Atlas	Database (DBaaS)	Primary data store for every collection in the system.
Cloudinary	Media storage (SaaS)	Profile photos, written-answer page photographs, exam-session recording chunks.
LiveKit	Real-time media (SFU)	Live camera/microphone/screen-share streaming and data-channel chat during proctored exams.

No payment gateway, email/SMS provider, or ticketing/CRM integration exists in either repository.

4.5 Deployment Targets

LAYER	PLATFORM	EVIDENCE
Frontend	Vercel	Confirmed — <code>vercel.json</code> present in the client repository with an SPA rewrite rule; the configured CORS origin on the backend also references a <code>*.vercel.app</code> hostname.
Backend	Inferred: Render	To be verified — no deployment manifest exists in the server repository; the client's <code>.env.example</code> contains a commented-out reference to an <code>onrender.com</code> URL. This is circumstantial, not a confirmed deployment configuration.

5. Application Modules

Modules here are organized by real business domain, not by folder name — several map to more than one backend NestJS module or frontend feature folder.

Authentication

Roles	All
Frontend	<code>features/auth/*</code> (login, forgot/reset password)
Backend	<code>AuthModule</code> , <code>AuthService</code> , <code>AuthJwtService</code> , <code>PasswordService</code>
APIs	6 endpoints under <code>/auth</code> (Section 10.1)
Database	<code>User</code> , <code>AuthActivityLog</code>
Business rules	Bcrypt password checks; forgot-password disabled for admin accounts; mandatory first-login password change.

Academic Management

Roles	Super Admin (full); read-only lookups feed exam authoring
Frontend	<code>features/super-admin/academics/*</code>
Backend	<code>AdminModule</code> hierarchy controllers
APIs	12 endpoints (Section 10.1)
Database	<code>Batch</code> , <code>Course</code> , <code>BatchCourse</code> , <code>Department</code> , <code>BatchDepartment</code> , <code>Section</code> , <code>Subject</code>
Business rules	A course/department/section can only be attached through its proper mapping chain — a section cannot exist without a valid batch-course-department mapping already in place.

Student Management

Roles	Super Admin (full); Student (self-serve profile only)
Frontend	<code>features/super-admin/students/*</code> , <code>features/student/profile/*</code>
Backend	<code>AdminModule</code> student-management, <code>StudentModule</code> profile
APIs	7 admin CRUD/activity endpoints + 2 student profile endpoints
Database	<code>Student</code> + its 6 detail collections (Section 12.2)
Business rules	Creation collects only identity + academic placement; address, emergency contact, education history, and parent/guardian details are completed by the student themselves later.

Faculty Management

Roles	Super Admin (full); Faculty (self-serve profile only)
Frontend	<code>features/super-admin/faculties/*</code> , <code>features/faculty/profile/*</code>
Backend	<code>AdminModule</code> faculty-management, <code>FacultyModule</code> profile
APIs	6 admin CRUD/activity endpoints + 2 faculty profile endpoints
Database	<code>Faculty</code> + its 6 detail collections
Business rules	Same minimal-creation, self-serve-completion pattern as Student Management.

Examination Management

Roles	Super Admin (author/publish); Faculty (evaluate); Student (take)
Frontend	<code>features/super-admin/exams/*</code> , <code>features/student/exams/*</code> , <code>features/faculty/evaluation/*</code>
Backend	<code>AdminModule</code> exam-management, <code>StudentModule</code> exam-management, <code>FacultyModule</code> exam-evaluation
APIs	20 admin + 17 student exam-taking + 3 faculty evaluation endpoints
Database	<code>Exam</code> , <code>ExamQuestion</code> , <code>ExamAttempt</code> , <code>ExamAnswer</code>
Business rules	See Section 16 (Examination System) in full.

Exam Room Management & Proctoring

Roles	Super Admin (form rooms, review); Faculty (invigilate); Student (participate)
Frontend	<code>features/faculty/proctoring/*</code> , <code>features/student/exams</code> (lobby/proctored-room pages)
Backend	<code>AdminModule</code> room formation, <code>FacultyModule</code> exam-proctoring, <code>StudentModule</code> exam-proctoring, <code>LiveKitModule</code>
APIs	Room formation + review (admin), 8 room-monitoring endpoints (faculty), 5 lobby/proctoring endpoints (student)
Database	<code>ExamRoom</code> , <code>ExamRoomAssignment</code> , <code>ExamRoomChatMessage</code>
External services	LiveKit
Business rules	See Section 17 (Proctoring Architecture) in full.

Evaluation & Results

Roles	Super Admin (assign evaluators, publish); Faculty (score)
Frontend	<code>features/faculty/evaluation/*</code>
Backend	<code>FacultyModule</code> exam-evaluation, <code>AdminModule</code> exam-management (publish-results, evaluators, evaluation-progress)
Database	<code>ExamAnswer</code> (marksAwarded, evaluatedBy, remarks), <code>Exam</code> (evaluatorFacultyIds)
Business rules	Only Written/Typing answers require manual evaluation; MCQ/MSQ are scored automatically at submission time; results can only be published once evaluation is complete for the exam.

Dashboard

Roles	Super Admin, Faculty, Student (each their own)
Frontend	<code>features/{super-admin,faculty,student}/dashboard/*</code>
Backend	<code>AdminModule</code> dashboard (1 endpoint); faculty/student dashboards compose existing per-role queries rather than a dedicated backend module.

File Management

Roles	Student (written-answer upload, recording upload)
Backend	<code>CloudinaryService</code> , consumed from <code>PublicModule</code> and <code>StudentModule</code>
Business rules	See Section 21 (File & Storage Architecture) in full.

Public / Marketing

Roles	Unauthenticated visitors
Frontend	<code>features/public/landing/*</code> , <code>features/public/documentation/*</code> , <code>features/public/written-answer/*</code>
Backend	<code>PublicModule</code> (written-answer only — the landing/documentation pages are static client-rendered content with no dedicated backend endpoints)

6. User Roles & Responsibilities

XaminiyIQ implements exactly three roles — `ADMIN` (Super Admin), `FACULTY`, and `STUDENT` — defined in the shared `UserRole` enum and enforced identically at the database level (every user document carries a role), the API level (every protected controller method declares `@Roles(...)`), and the frontend routing level (every dashboard route is wrapped in a matching `RoleGuard`).

7.1 Super Admin

Purpose: Owns the institution's academic setup and the exam lifecycle end to end.

RESPONSIBILITIES

- Create and maintain the academic hierarchy: batches, courses, batch-course mappings, departments, batch-department mappings, sections, subjects.
- Onboard students and faculty (individually or via bulk CSV), edit or deactivate accounts, review activity history.
- Author exams — configure hierarchy targeting, schedule, question sets, security settings — and publish them.
- Form proctoring rooms and trigger invigilator assignment for PROCTORING exams.
- Review assigned students per exam, exam attempts, recordings, and per-attempt answers across all question types.
- Assign faculty evaluators to an exam and publish results once evaluation is complete.

ACCESSIBLE MODULES

Full access to Academic Management, User Management (Students, Faculty), Examination Management, Exam Room Allocation, and the Super Admin Dashboard.

RESTRICTIONS

Cannot invigilate a live proctoring session directly (that surface is faculty-only) and cannot answer exam questions as a student.

7.2 Faculty

Purpose: Owns subject content ownership and live exam invigilation/evaluation.

RESPONSIBILITIES

- Manage subjects they own.
- Invigilate rooms they are assigned to: admit or reject waiting students, view each admitted student's live camera and screen-share feed, mute a student's microphone, chat with students individually or by broadcast, and remove a student mid-exam (finalizing their attempt).
- Evaluate Written and Typing answers for exams they are an assigned evaluator on — award marks and remarks per question.

ACCESSIBLE MODULES

Faculty Dashboard, their own Profile, Subjects (their own), Proctoring (rooms they invigilate), Evaluation (exams they are an assigned evaluator for).

RESTRICTIONS

Cannot create/edit exams, cannot manage the academic hierarchy, cannot manage other users, cannot view a room they are not assigned to (enforced server-side — every proctoring endpoint checks the room's `facultyId` against the calling faculty before returning data).

7.3 Student

Purpose: Completes their profile and takes the exams they are eligible for.

RESPONSIBILITIES

- Complete their profile (address, emergency contact, education history, parent/guardian, photo) after first login — not required at account-creation time.
- View subjects and exams matching their own batch/course/department/section/semester.
- Run the pre-flight environment check before any exam attempt.
- For PROCTORING exams: join the waiting room and wait for faculty admission.
- Answer MCQ/MSQ questions, type Typing answers, or capture Written answers via the QR/phone-photo flow.
- Submit the exam (manually, or automatically at time-up / on repeated integrity violations) and view results once published.

ACCESSIBLE MODULES

Student Dashboard, their own Profile, Subjects (their own), My Exams, the exam-taking room, and their own exam results.

RESTRICTIONS

Can only see and act on exams matching their own academic placement (enforced server-side on every exam-attempt endpoint, not only in the listing query) and can never see another student's attempt, answers, or proctoring feed.

7.4 Role × Module Access Matrix

MODULE	SUPER ADMIN	FACULTY	
	STUDENT		
Academic Management (batches/courses/department s/sections/subjects)	Full	X	X
Student Management	Full	X	Self only (profile completion)

Faculty Management	Full	Self only (profile completion)	X
Exam Authoring & Publishing	Full	X	X
Exam Room Formation	Full	X	X
Proctoring (live monitoring)	Read-only review (recordings, answers)	Full, scoped to assigned rooms	Participant (own session only)
Exam Taking	X	X	Full, scoped to eligible exams
Evaluation (Written/Typing scoring)	Assigns evaluators; publishes results	Scores answers, scoped to assigned exams	X
Dashboard	Admin overview	Faculty overview	Student overview

This matrix reflects the actual authorization implementation — every "Full"/"scoped" claim corresponds to a real `@Roles()` guard and, where noted, an additional ownership check inside the service method (e.g. `ownRoomOrThrow` , `ownAttemptOrThrow`) rather than role alone.

7. Application Workflows

Authentication is diagrammed in Section 18.1; the remaining core workflows are shown here.

7.1 Student Onboarding Workflow

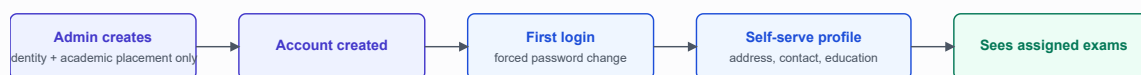


Figure 7.1 — Student onboarding. Bulk CSV upload follows the same path for many students at once, one row per student, with a per-row error report.

7.2 Faculty Onboarding Workflow

Identical shape to Student Onboarding (Figure 7.1): Admin creates a minimal faculty account (identity + employment placement) → first login forces a password change → the faculty completes their own address, emergency contact, education history, and work experience via the self-serve profile flow → they can then be assigned subjects and, later, invigilation rooms. There is no bulk-upload path for faculty (individual creation only).

7.3 Exam Creation Workflow

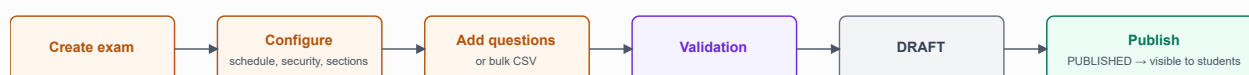


Figure 7.2 — Exam creation. Validation at each stage is re-run at publish time too (total marks must match the sum of question marks; passing marks must not exceed total marks) — a draft can be saved incomplete, but cannot be published while inconsistent.

For PROCTORING exams, publishing is followed by a separate, explicit admin action: **Form Exam Rooms** (Section 17.2), which is not part of the publish step itself — an exam can sit in **PUBLISHED** with no rooms formed yet.

7.4 Student Examination Workflow (AUTO Mode)

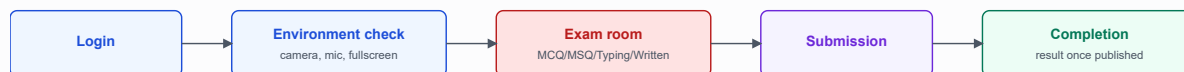


Figure 7.3 — AUTO-mode exam. No waiting room or faculty approval step exists for this mode — the environment check leads directly into the exam room.

7.5 Student Examination Workflow (PROCTORING Mode)

Identical to Figure 7.3 but with **Waiting Room → Faculty Approval** inserted between the environment check and the exam room, and continuous live monitoring throughout the exam room stage — the full sequence and monitored conditions are documented in Section 17 (Proctoring Architecture) and Section 7.6 below.

7.6 Proctoring Workflow

See Figure 17.1 (Proctoring Architecture, Section 17) for the component topology and Section 17.2 for the full session-lifecycle narrative (room formation → waiting room → admission → live session → disconnect handling).

8. Frontend Architecture

8.1 Application Structure

The client is a single Vite-built React 19 SPA. `src/main.tsx` mounts `App.tsx`, which renders the route tree via `src/app/providers/AppProvider.tsx` (Redux `<Provider>` + a global `react-hot-toast` `<Toaster>`) and `src/app/routes/MainRoutes.tsx`.

Folder structure (`src/`)

<code>src/</code>	
├─ <code>api/</code>	Plain Axios API modules (auth, academics, exams, ...) – used where RTK Query isn't applicable, e.g. the unauthenticated mobile written-answer flow
├─ <code>app/</code>	
│ └─ <code>providers/</code>	<code>AppProvider.tsx</code> – Redux Provider + Toaster
│ └─ <code>routes/</code>	<code>MainRoutes</code> , <code>AuthRoutes</code> , <code>PublicRoutes</code> , <code>ProtectedRoutes</code> , and per-role route files
│ └─ <code>store/</code>	<code>store.ts</code> , <code>rootReducer.ts</code> , <code>hooks.ts</code>
├─ <code>assets/</code>	fonts, gifs, icons, images
├─ <code>common/</code>	
│ └─ <code>form-sections/</code>	Shared Formik field groups reused across Create/Edit pages
│ └─ <code>loaders/</code>	
│ └─ <code>ui/</code>	17 reusable presentational components (Section 8.5)
├─ <code>config/</code>	<code>env.ts</code> – the single <code>import.meta.env</code> access point
├─ <code>features/</code>	
│ └─ <code>auth/</code>	login, logout, forgot-password, reset-password
│ └─ <code>common/</code>	page-not-found, un-authorized
│ └─ <code>faculty/</code>	dashboard, evaluation, proctoring, profile, subjects, routes
│ └─ <code>public/</code>	documentation, landing, written-answer (QR mobile capture)
│ └─ <code>student/</code>	dashboard, exams, profile, subjects, routes
│ └─ <code>super-admin/</code>	academics, exams, faculties, students, dashboard, routes
├─ <code>hoc/</code>	<code>withAuthGuard</code> , <code>withGuestGuard</code> , <code>withRoleGuard</code> , <code>withScreenGuard</code>
├─ <code>layouts/</code>	auth, exam-room, public, root layout shells
├─ <code>state/</code>	
│ └─ <code>reducers/</code>	<code>auth.slice.ts</code> (present but not wired into the store – see 10.3)
│ └─ <code>services/</code>	<code>api-instance.ts</code> , <code>axios-instance.ts</code> , <code>base-query.ts</code> , <code>endpoints/</code> (17 files)
├─ <code>types/</code>	Per-domain TypeScript interfaces (12 files)
└─ <code>utils/</code>	permissions, constants, navigation, storage, date, enum, sanitizeHtml, etc.

8.2 Routing Architecture

`MainRoutes.tsx` concatenates four route groups plus two catch-alls:

```
routes = [...authRoutes, ...publicRoutes, ...protectedRoutes, "/unauthorized", "* (404)"]
```

Two composed, guarded layouts wrap the protected routes:

```
ProtectedRootLayout = withAuthGuard(withScreenGuard(RootLayout))
```

```
ExamRoomProtectedLayout = withAuthGuard(withScreenGuard(ExamRoomLayout))
```

RootLayout renders the shared sidebar/header shell (role-driven navigation config) for all Super Admin, Faculty, and Student dashboard pages. **ExamRoomLayout** is a deliberately bare `<Outlet />` — no sidebar/header — used for the pre-flight check, lobby, exam room, and faculty proctoring dashboard pages, so nothing distracts from (or is exposed during) an active exam or proctoring session. Both layouts still enforce authentication and the $\geq 768\text{px}$ screen-width guard before rendering their outlet.

Complete route table

PATH	PAGE	LAYOUT	GUARD
/login	LoginPage	AuthLayout	Guest-only
/forgot-password	ForgotPasswordPage	AuthLayout	Guest-only
/reset-password	ResetPassword	AuthLayout	Guest-only
/reset-password/:token	ResetPasswordPage	AuthLayout	Guest-only
/	LandingPage	PublicLayout	Guest-only
/documentation	DocumentationPage	PublicLayout	None
/mobile/written-answer/:token	MobileWrittenAnswerPage	None (bare)	None — QR token is the credential
/super-admin/dashboard	SuperAdminDashboardPage	RootLayout	ADMIN
/super-admin/academics/*	Batches / Courses / Departments / Sections / Subjects pages	RootLayout	ADMIN
/super-admin/students, /create, /:id, /:id/edit	Students CRUD pages	RootLayout	ADMIN
/super-admin/faculties, /create, /:id, /:id/edit	Faculties CRUD pages	RootLayout	ADMIN
/super-admin/exams, /create, /:id, /:id/edit	Exams CRUD pages	RootLayout	ADMIN
/faculty/dashboard	FacultyDashboardPage	RootLayout	FACULTY
/faculty/profile	FacultyProfilePage	RootLayout	FACULTY
/faculty/subjects	SubjectsPage	RootLayout	FACULTY
/faculty/proctoring	MyExamRoomsPage	RootLayout	FACULTY
/faculty/proctoring/:roomId	ProctoringDashboardPage	ExamRoomLayout	FACULTY
/faculty/evaluation, /:examId	MyEvaluationExamsPage / EvaluationQueuePage	RootLayout	FACULTY
/student/dashboard	StudentDashboardPage	RootLayout	STUDENT
/student/profile	StudentProfilePage	RootLayout	STUDENT

/student/subjects	SubjectsPage	RootLayout	STUDENT
/student/exams	MyExamsPage	RootLayout	STUDENT
/student/exams/results/:attemptId	ExamResultPage	RootLayout	STUDENT
/student/exams/:examId/pre-flight	PreFlightCheckPage	ExamRoomLayout	STUDENT
/student/exams/:examId/lobby	LobbyPage	ExamRoomLayout	STUDENT
/student/exams/:examId/room/:attemptId	ExamRoomPage (AUTO)	ExamRoomLayout	STUDENT
/student/exams/:examId/proctored-room/:attemptId	ProctoredExamRoomPage	ExamRoomLayout	STUDENT
/unauthorized	UnAuthorizedPage	None	None
*	PageNotFound	None	None

Route guards (`src/hoc/`)

GUARD	TYPE	BEHAVIOR
withAuthGuard	HOC	Checks for a stored access token; redirects to <code>/login</code> if absent.
withGuestGuard	HOC	Inverse — if an access token exists, redirects an already-authenticated user away from auth/landing pages to their role's dashboard.
RoleGuard (withRoleGuard)	Component	Takes <code>allowedRoles</code> ; compares against the stored <code>userRole</code> ; redirects to <code>/unauthorized</code> on mismatch.
withScreenGuard	HOC	Tracks viewport width; below 768px it renders a blocking "optimized for larger screens" message instead of the wrapped layout — no redirect, a substitute render.

8.3 State Management

The Redux store (`configureStore`) combines exactly one reducer slice — `apiInstance.reducer` (RTK Query's own cache) — plus its middleware. An `authSlice` exists in the codebase but is commented out of the root reducer: authenticated-user state (access token, role, user id) lives in `localStorage` via `src/utils/storage.ts` , not Redux. Redux Toolkit is therefore used almost entirely as RTK Query's cache/middleware host, not as a general application-state store.

RTK Query is configured with a custom Axios-backed `baseQuery` (`src/state/services/base-query.ts`) instead of the default `fetch` -based one, so every request goes through the same Axios instance and its interceptor

pipeline (`src/state/services/axios-instance.ts`):

- **Request interceptor** decodes the JWT payload client-side to read its expiry, and proactively calls `POST /auth/refresh-token` (with `withCredentials: true`, so the httpOnly refresh cookie is sent) when the access token is within a small grace window of expiring. Requests issued while a refresh is already in flight are queued and replayed once the new token arrives.
- **Response interceptor** performs the same refresh-and-retry reactively on a 401, skipped for a short list of public endpoints (login, refresh, forgot/reset password). A 403 redirects to `/unauthorized`; a failed refresh clears stored auth state and redirects to `/login`.

API domains are split across 17 files in `src/state/services/endpoints/` (auth, academics, students, faculty, exams, subjects, evaluation, dashboard, student-exams, student-profile, faculty-profile, faculty-proctoring, student-proctoring, written-answer, ...), each using RTK Query's `injectEndpoints` against the single shared `apiInstance`. Cache invalidation is tag-based, using a fixed set of tag types declared once centrally (`batches`, `students`, `exam-detail`, `assigned-students`, `my-student-profile`, and others — 23 tags in total).

8.4 Form Handling & Validation

Every data-entry form in the application uses the same pattern: Formik for form state/submission, and a sibling Yup schema in a `formik/` folder next to the page (e.g. `features/super-admin/exams/formik/create-exam.schema.ts`). There are 14 such schema files across the client. Validation errors surface inline per-field; submission failures surface as a toast (`react-hot-toast`) built from the API's error response message.

8.5 Reusable UI Component Library (`src/common/ui/`)

COMPONENT	PURPOSE
Button	5 variants (primary/secondary/outline/danger/ghost), 3 sizes, loading/icon support.
Modal / DeleteConfirm Modal	Generic dialog and a pre-built delete-confirmation dialog built on it.
Table	Generic data table — sorting, search, pagination, typed column definitions.
Select / AsyncSelect / MultiSelect	Static, paginated-async, and multi-value dropdowns (the latter two wrap <code>react-select</code>).
Input	Text input with label, icon, and password-visibility toggle.
Chip	Colored status badge, 8 color variants.
Container / PageHeader	Page-level layout wrappers.
Accordion	Collapsible section, used in question authoring and detail views.
Timeline	Vertical timeline renderer for education/work-experience history.
Chart	Thin ECharts lifecycle wrapper (init/dispose/resize).

CountdownTimer	HH:MM:SS countdown, used on the exam-room and proctoring pages.
ProfileCompletionBar	Self-serve profile completion % indicator with a CTA.
RowActions	Table-row action icon cluster (edit/delete/view-activity).

8.6 Responsive Design Approach

Layout responsiveness follows Tailwind's default breakpoint scale throughout (`sm:` / `md:` / `lg:` utility variants), with one deliberate exception: `withScreenGuard` enforces a hard 768px minimum for every authenticated dashboard/exam-room route, since exam authoring, proctoring monitoring, and exam-taking are not considered usable on a small-screen device. The public landing page and the QR-scanned mobile written-answer page are the two intentionally mobile-first surfaces in the application.

9. Backend Architecture

9.1 Framework & Module Structure

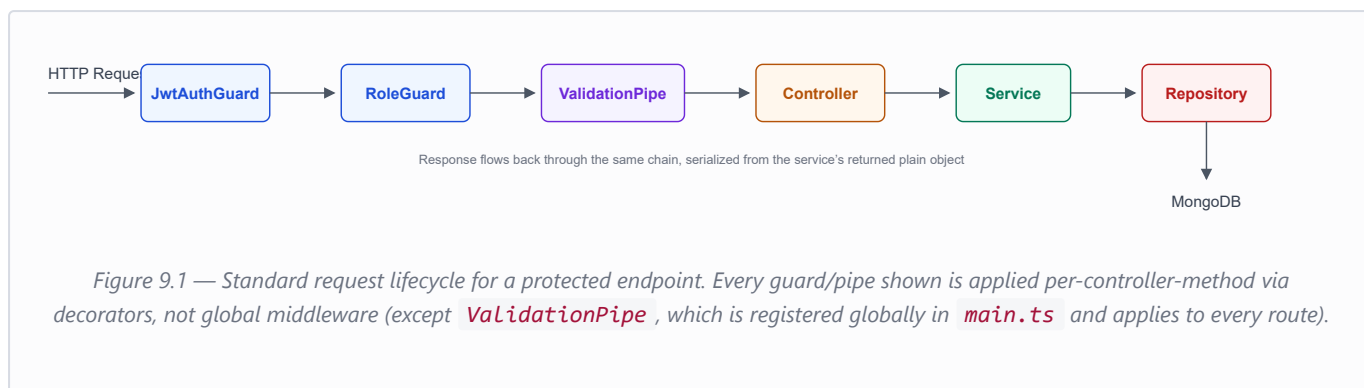
The backend is a NestJS 11 application. Six top-level feature modules compose the app: `AuthModule`, `PublicModule` (unauthenticated written-answer QR flow), `AdminModule`, `FacultyModule`, `StudentModule`, and `LiveKitModule` (webhook + real-time services). Two shared modules underpin all of them:

- `RepositoryModule` registers every Mongoose schema via `MongooseModule.forFeature` and exports one repository service per collection (31 repositories, one per schema in Section 12).
- `ServiceModule` imports `RepositoryModule` and exports every business-logic service, so a feature module only ever imports `ServiceModule` to reach the full data-access layer.

Folder structure (`src/`)

```
src/
├── api/                               Controllers + request/response DTOs (one file per class, one class per
file)
├──   ├── auth/                       login, logout, refresh-token, change/forgot/reset-password
├──   ├── public/written-answer/      Unauthenticated QR-based mobile capture endpoints
├──   └── user/
├──       ├── admin/                 45 endpoints: exam-management/, faculty-management/, student-management/,
├──       │                       dashboard/, plus academic-hierarchy lookups
├──       ├── faculty/               19 endpoints: exam-evaluation/, exam-proctoring/, profile/, subject-
management/
├──       └── student/               24 endpoints: exam-management/, exam-proctoring/, profile/, subject-
management/
├── services/                         Business logic – one service class per domain area
├──   ├── auth-service/              auth.service.ts, jwt.service.ts, password.service.ts
├──   ├── public-service/            written-answer.service.ts
├──   ├── scheduler-service/         exam-lifecycle-scheduler.service.ts
├──   └── user-service/{admin,faculty,student}/
├── repositories/                   31 folders – one Mongoose data-access class per collection
├── schemas/                       Mongoose model definitions (Academic/, Exam/,
User/{Admin,Faculty,Student}/, AuthActivityLog/)
├── guards/                         jwt-auth.guard.ts, role.guard.ts
├── decorators/                     roles.decorator.ts, current-user.decorator.ts
├── livekit/                         livekit.service.ts, livekit-webhook.controller.ts/.service.ts, livekit-
disconnect-sweep.service.ts
├── cloudinary/                     cloudinary.service.ts
├── config/                         config.service.ts – the single environment-variable access point
├── database/seeders/               Initial-admin and academic-reference-data seed scripts
├── utils/                          cookie.util.ts, date.util.ts, enum.ts, utils.ts
└── main.ts                        Bootstrap: CORS, ValidationPipe, cookie-parser, raw-body for webhooks
```

9.2 Request Lifecycle



Stage-by-stage

STAGE	RESPONSIBILITY
JwtAuthGuard	Extracts the Bearer token, verifies it against <code>JWT_SECRET_KEY</code> , and attaches the decoded payload to <code>request.user</code> . Throws <code>UnauthorizedException</code> on a missing/invalid token.
RoleGuard	Reads the <code>@Roles()</code> metadata off the handler; if present, requires <code>request.user.role</code> to be in the allowed set, else throws <code>ForbiddenException</code> . A method with no <code>@Roles()</code> decorator passes through unconditionally (used by <code>/auth/logout</code> , which only needs a valid session, not a specific role).
ValidationPipe (global)	<code>whitelist: true, forbidNonWhitelisted: true, transform: true</code> — strips any property not declared on the DTO, rejects the request outright if an undeclared property is present, and coerces primitive types (e.g. query-string numbers) to their declared type.
Controller	Thin — extracts the authenticated user id from <code>request.user.sub</code> , delegates to exactly one service method, and shapes the return value into the endpoint's declared <code>*Response</code> class.
Service	All business rules live here: hierarchy validation, eligibility checks, state-machine transitions, cross-collection aggregation.
Repository	The only layer that imports a Mongoose model directly; every method maps 1:1 to a specific query shape a service needs (no generic "find" passthrough).

9.3 Error Handling

NestJS's built-in HTTP exception classes are used directly by services (`NotFoundException`, `BadRequestException`, `ConflictException`, `ForbiddenException`, `UnauthorizedException`) rather than a custom exception hierarchy — each carries a human-readable message that becomes the API error response's `message` field. Repository methods catch low-level driver errors and re-throw as `InternalServerErrorException` with the underlying message attached, so a database-layer failure never leaks a raw Mongoose/MongoDB stack trace to the client. See Section 10.4 for the standard error-response shape.

9.4 Business Logic Highlights

Two service methods are worth calling out specifically for their complexity, since they are the most consequential pieces of logic in the system:

- `formExamRoomsAPI` (`exam-management.service.ts`) — resolves every student matching the exam's target hierarchy, discovers other unformed PROCTORING exams sharing the exact same schedule window, pools their small remainders together, partitions the combined student set into groups of at most 10, and round-robin assigns an active faculty member per group — all inside a single method, described in full in Section 17.2.
- `ExamLifecycleSchedulerService` (Section 24) — the only place in the backend where state changes without any incoming HTTP request, driving the exam status state machine purely from wall-clock time.

10. API Documentation

The backend exposes 98 REST endpoints (no global path prefix), plus one unauthenticated LiveKit webhook. Every endpoint under `/admin`, `/faculty`, and `/student` uses the identical guard stack

`@UseGuards(JwtAuthGuard, RoleGuard)` with exactly one `@Roles(...)` value per method — there are no exceptions to this pattern anywhere in the codebase. `/auth/*` and `/public/written-answer/*` are the only genuinely unauthenticated REST surfaces.

10.1 Complete Endpoint Reference

Authentication (`/auth`) — 6 endpoints, public

METHOD	PATH	ACCESS	REQUEST	RESPONSE
POST	/auth/login	Public	LoginRequest	LoginResponse
POST	/auth/logout	Authenticated (any role)	—	LogoutResponse
POST	/auth/refresh-token	Public (httpOnly cookie)	—	RefreshTokenResponse
POST	/auth/change-password	Public (first-login flow)	ChangePasswordRequest	ChangePasswordResponse
POST	/auth/forgot-password	Public	ForgotPasswordRequest	ForgotPasswordResponse
POST	/auth/reset-password	Public	ResetPasswordRequest	ResetPasswordResponse

Public — Written Answer Capture (`/public/written-answer`) — 3 endpoints

METHOD	PATH	ACCESS	REQUEST	RESPONSE
POST	/public/written-answer/verify	Public — QR token is the credential	VerifyQrTokenRequest	VerifyQrTokenResponse
POST	/public/written-answer/signature	Public — QR token	GetUploadSignatureRequest	GetUploadSignatureResponse
POST	/public/written-answer/page	Public — QR token	RecordPageRequest	RecordPageResponse

Webhook — `/webhooks/livekit` — 1 endpoint

METHOD	PATH	ACCESS	REQUEST	RESPONSE
--------	------	--------	---------	----------

POST	/webhooks/livekit	Public — HMAC-signed raw body verified against the LiveKit API secret	raw body	<code>{received: boolean}</code>
-------------	-------------------	---	----------	----------------------------------

Super Admin — Academic Hierarchy (/admin) — 12 endpoints, **ADMIN**

METHOD	PATH	REQUEST	RESPONSE
POST	/admin/batches	CreateBatchRequest	CreateBatchResponse
GET	/admin/batches	GetAllBatchesRequest (query)	GetAllBatchesResponse
POST	/admin/batches/:batchId/courses	MapCourseToBatchRequest	MapCourseToBatchResponse
GET	/admin/batches/:batchId/courses	GetAllCoursesForBatchRequest (query)	GetAllCoursesForBatchResponse
GET	/admin/batches/:batchId/available-courses	—	GetCoursesByBatchResponse
POST	/admin/batch-courses/:batchCourseId/departments	AddDepartmentToBatchCourseRequest	AddDepartmentToBatchCourseResponse
GET	/admin/batch-courses/:batchCourseId/departments	GetAllDepartmentForBatchCourseRequest (query)	GetAllDepartmentForBatchCourseResponse
GET	/admin/batch-departments/:batchDepartmentId/sections	—	GetDepartmentSectionsResponse
GET	/admin/courses-with-departments	GetAllCoursesWithDepartmentsRequest (query)	GetAllCoursesWithDepartmentsResponse
GET	/admin/courses/:courseId/departments	—	GetDepartmentsByCourseResponse
GET	/admin/departments	—	GetAllDepartmentsResponse
GET	/admin/subjects	GetAllSubjectsRequest (query)	GetAllSubjectsResponse

Super Admin — Exam Management (/admin) — 20 endpoints, **ADMIN**

METHOD	PATH	REQUEST	RESPONSE
POST	/admin/exams	CreateExamRequest	CreateExamResponse
GET	/admin/exams	GetAllExamsRequest (query)	GetAllExamsResponse
GET	/admin/exams/:id	—	GetExamResponse
PATCH	/admin/exams/:id	EditExamRequest	EditExamResponse

DELETE	/admin/exams/:id	—	DeleteExamResponse
PATCH	/admin/exams/:id/publish	—	PublishExamResponse
POST	/admin/exams/:id/questions	AddQuestionRequest	AddQuestionResponse
POST	/admin/exams/:id/questions/bulk-upload	BulkUploadQuestionsRequest	BulkUploadQuestionsResponse
PATCH	/admin/exams/:id/questions/:questionId	EditQuestionRequest	EditQuestionResponse
DELETE	/admin/exams/:id/questions/:questionId	—	DeleteQuestionResponse
POST	/admin/exams/:id/rooms	—	FormExamRoomsResponse
GET	/admin/exams/:id/rooms	—	GetExamRoomsResponse
GET	/admin/exam-rooms	GetAllExamRoomsRequest (query)	GetAllExamRoomsResponse
PATCH	/admin/exams/:id/evaluators	AssignEvaluatorsRequest	AssignEvaluatorsResponse
GET	/admin/exams/:id/evaluation-progress	—	GetEvaluationProgressResponse
POST	/admin/exams/:id/publish-results	—	PublishResultsResponse
GET	/admin/exams/:id/attempts	—	GetExamAttemptsResponse
GET	/admin/exams/:id/assigned-students	—	GetAssignedStudentsResponse
GET	/admin/attempts/:attemptId/answers	—	GetAttemptAnswersResponse
GET	/admin/attempts/:attemptId/recording	—	GetAttemptRecordingResponse

Super Admin — Student / Faculty Management & Dashboard (/admin) — 14 endpoints, **ADMIN**

METHOD	PATH	REQUEST	RESPONSE
POST	/admin/students	CreateStudentRequest	CreateStudentResponse
GET	/admin/students	Query filters	GetAllStudentsResponse
GET	/admin/students/:id	—	GetStudentResponse
PATCH	/admin/students/:id	EditStudentRequest	EditStudentResponse
DELETE	/admin/students/:id	—	DeleteStudentResponse
GET	/admin/students/:id/activity	—	GetStudentActivityResponse
POST	/admin/students/bulk-upload	BulkUploadStudentsRequest	BulkUploadStudentsResponse
POST	/admin/faculty	CreateFacultyRequest	CreateFacultyResponse
GET	/admin/faculty	Query filters	GetAllFacultyResponse
GET	/admin/faculty/:id	—	GetFacultyResponse

PATCH	/admin/faculty/:id	EditFacultyRequest	EditFacultyResponse
DELETE	/admin/faculty/:id	—	DeleteFacultyResponse
GET	/admin/faculty/:id/activity	—	GetFacultyActivityResponse
GET	/admin/dashboard/overview	—	GetDashboardOverviewResponse

Faculty (/faculty) — 19 endpoints, **FACULTY**

METHOD	PATH	REQUEST	RESPONSE
GET	/faculty/evaluation/exams	—	GetMyEvaluationExamsResponse
GET	/faculty/evaluation/exams/:examId/answers	—	GetExamAnswersForEvaluationResponse
PATCH	/faculty/evaluation/answers/:answerId	EvaluateAnswerRequest	EvaluateAnswerResponse
GET	/faculty/exam-rooms	—	GetMyExamRoomsResponse
GET	/faculty/exam-rooms/:roomId	—	GetExamRoomDetailResponse
POST	/faculty/exam-rooms/:roomId/assignments/:assignmentId/admit	—	AdmitStudentResponse
POST	/faculty/exam-rooms/:roomId/assignments/:assignmentId/reject	RejectStudentRequest	RejectStudentResponse
POST	/faculty/exam-rooms/:roomId/assignments/:assignmentId/remove	RemoveStudentRequest	RemoveStudentResponse
POST	/faculty/exam-rooms/:roomId/assignments/:assignmentId/mic	SetStudentMicRequest	SetStudentMicResponse
POST	/faculty/exam-rooms/:roomId/chat	SendChatRequest	SendChatResponse
GET	/faculty/exam-rooms/:roomId/chat	—	GetChatHistoryResponse
POST	/faculty/exam-rooms/:roomId/livekit-token	—	GetLiveKitTokenResponse
GET	/faculty/profile	—	GetMyProfileResponse
PATCH	/faculty/profile	UpdateMyFacultyProfileRequest	UpdateMyProfileResponse
POST	/faculty/subjects	CreateSubjectRequest	CreateSubjectResponse
GET	/faculty/subjects	Query filters	GetAllSubjectsResponse
GET	/faculty/subjects/:id	—	GetSubjectResponse
PATCH	/faculty/subjects/:id	EditSubjectRequest	EditSubjectResponse
DELETE	/faculty/subjects/:id	—	DeleteSubjectResponse

Student (/student) — 24 endpoints, **STUDENT**

METHOD	PATH	REQUEST	RESPONSE
GET	/student/exams	—	GetAllExamsResponse
POST	/student/exams/:examId/start	—	StartAttemptResponse
GET	/student/exams/attempts/:attemptId	—	GetAttemptResponse
PATCH	/student/exams/attempts/:attemptId/answers/:questionId	SaveAnswerRequest	SaveAnswerResponse
POST	/student/exams/attempts/:attemptId/questions/:questionId/view	—	ViewQuestionResponse
POST	/student/exams/attempts/:attemptId/submit	SubmitAttemptRequest	SubmitAttemptResponse
POST	/student/exams/attempts/:attemptId/violations	ReportViolationRequest	ReportViolationResponse
POST	/student/exams/attempts/:attemptId/recordings/signature	RecordingSignatureRequest	RecordingSignatureResponse
POST	/student/exams/attempts/:attemptId/recordings/chunk	RecordChunkRequest	RecordChunkResponse
POST	/student/exams/attempts/:attemptId/recordings/finalize	FinalizeRecordingRequest	FinalizeRecordingResponse
GET	/student/exams/network-probe/download	—	raw octet-stream
POST	/student/exams/network-probe/upload	raw stream	{success,message}
POST	/student/exams/attempts/:attemptId/answers/:questionId/qr	—	GenerateWrittenQrResponse
GET	/student/exams/attempts/:attemptId/answers/:questionId/qr-status	—	GetWrittenQrStatusResponse
POST	/student/exams/attempts/:attemptId/answers/:questionId/finalize	—	FinalizeWrittenAnswerResponse
DELETE	/student/exams/attempts/:attemptId/answers/:questionId/pages/:pageNumber	—	DeleteWrittenAnswerPageResponse
GET	/student/exams/attempts/:attemptId/result	—	GetMyResultResponse
POST	/student/exam-rooms/:examId/join	—	JoinLobbyResponse
GET	/student/exam-rooms/lobby/:assignmentId	—	GetLobbyStatusResponse
POST	/student/exams/attempts/:attemptId/livekit-token	—	GetLiveKitTokenResponse
POST	/student/exam-rooms/:roomId/chat	SendChatRequest	SendChatResponse
GET	/student/exam-rooms/:roomId/chat	—	GetChatHistoryResponse
GET	/student/profile	—	GetMyProfileResponse
PATCH	/student/profile	UpdateMyStudentProfileRequest	UpdateMyProfileResponse
GET	/student/subjects	—	GetAllSubjectsResponse

IMPLEMENTATION NOTE

Two request DTO classes (`GetFacultyRequest` , `GetStudentRequest`) are defined in the codebase but never actually consumed by their controllers, which read the `:id` path parameter directly instead — dead code, not a documentation error. The two `network-probe` endpoints are the only ones in the entire API that work with raw binary streams instead of typed JSON DTOs, by design (they exist purely to measure client bandwidth for the pre-flight check).

10.2 Representative API Detail

The following endpoints are documented to full detail as representative examples of each major workflow. The complete request/response DTO shapes for every endpoint are defined in the corresponding `*.request.ts` / `*.response.ts` files in the backend repository, following the same per-field validation conventions shown here.

POST /auth/login**PURPOSE**

Authenticates a user by email/password and issues an access token (response body) and refresh token (httpOnly cookie).

AUTHENTICATION REQUIRED

No

REQUEST BODY

```
{
  "email": "admin@xaminityiq.edu",
  "password": "*****"
}
```

VALIDATION RULES

email : required, valid email format. **password** : required, string.

BUSINESS RULES

User must exist and be active; password is checked with **bcrypt.compare** against the stored hash. On success, both tokens are persisted onto the **User** document (not only returned to the client) and an **AuthActivityLog** entry is recorded with the request's IP and user-agent.

SUCCESS RESPONSE — 200

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "accessToken": "eyJhbGciOi...",
    "user": { "id": "665f...", "email": "admin@xaminityiq.edu", "role": "ADMIN", "isFirstLogin":
false }
  }
}
```

ERROR RESPONSES

401 invalid credentials. **403** account inactive.

DATABASE

User , **AuthActivityLog**

EXTERNAL SERVICES

None

POST /admin/exams**PURPOSE**

Creates a new exam in **DRAFT** status against a validated academic hierarchy selection.

AUTHENTICATION / AUTHORIZATION

Required — **ADMIN**

REQUEST BODY (ABRIDGED)

```
{
  "name": "Data Structures – Mid Semester",
  "mode": "PROCTORING",
  "batchId": "665f...", "courseId": "665f...", "departmentId": "665f...",
  "sectionIds": ["665f...", "665f..."],
  "semesters": [3, 4],
  "subjectId": "665f...",
  "durationMinutes": 60, "totalMarks": 50, "passingMarks": 20,
  "startDate": "2026-09-10", "endDate": "2026-09-10",
  "startTime": "09:00", "endTime": "10:30",
  "examSections": [{ "label": "Section A", "order": 1 }],
  "securitySettings": { "shuffleQuestions": true, "tabSwitchViolationThreshold": 3 }
}
```

VALIDATION RULES

Every hierarchy id must reference an existing, correctly-mapped entity (batch↔course↔department↔section chain validated end to end); **sectionIds** and **semesters** each require at least one entry; **startTime** / **endTime** are required when **mode = PROCTORING** and rejected/ignored for **AUTO** ; **passingMarks ≤ totalMarks** ; the scheduled window must be at least as long as **durationMinutes** .

SUCCESS RESPONSE — 201

```
{ "success": true, "message": "Exam created successfully", "examId": "665f..." }
```

ERROR RESPONSES

400 hierarchy/schedule validation failure. **404** a referenced hierarchy id does not exist.

DATABASE

Exam (also reads **Batch** , **Course** , **BatchCourse** , **Department** , **BatchDepartment** , **Section** , **Subject** for validation)

RELATED WORKFLOW

Exam Creation Workflow (Section 7.3)

POST /admin/exams/:id/rooms**PURPOSE**

Forms proctoring rooms for a PROCTORING exam — partitions matched students into rooms of at most 10 and round-robin assigns active faculty as invigilators.

AUTHENTICATION / AUTHORIZATION

Required — **ADMIN**

BUSINESS RULES

Exam must be `mode = PROCTORING` and `status = PUBLISHED`; rooms cannot already exist for this exam; at least one active faculty member must exist; other unformed PROCTORING exams sharing the exact same schedule window are pooled in automatically (Section 17.2).

SUCCESS RESPONSE — 201

```
{
  "success": true,
  "message": "Exam rooms formed successfully",
  "data": {
    "rooms": [{ "roomId": "665f...", "facultyId": "665f...", "facultyCode": "FAC1024",
  "studentCount": 9, "pooledExamNames": [] }],
    "warning": null
  }
}
```

BUSINESS NOTE

If the number of active faculty is smaller than the number of rooms formed, the same faculty is assigned to more than one room; the response's `warning` field is populated in that case so the client can surface it to the admin instead of it happening silently.

ERROR RESPONSES

400 not PROCTORING, not PUBLISHED, or no matching students/faculty. **409** rooms already formed.

DATABASE

`Exam`, `ExamRoom` (created), `ExamRoomAssignment` (created, one per matched student), `Faculty`, `Student`, `StudentAcademicDetail`

RELATED WORKFLOW

Proctoring Workflow (Section 7.6)

POST /student/exam-rooms/:examId/join**PURPOSE**

Joins the calling student's own waiting room for a PROCTORING exam they are assigned to.

AUTHENTICATION / AUTHORIZATION

Required — **STUDENT**

BUSINESS RULES

Exam must be **PROCTORING**, **PUBLISHED**, or **ONGOING**; the student's own batch/course/department/section/semester must match the exam's target hierarchy; a room assignment must already exist (i.e. an admin has run room formation). A student rejoining after a grace-period disconnect (status **DISCONNECTED**) is reset back to **WAITING**, not treated as a new join — **REJECTED** / **REMOVED** remain terminal.

SUCCESS RESPONSE — 200

```
{
  "success": true,
  "message": "Joined waiting room",
  "data": { "roomId": "665f...", "assignmentId": "665f...", "status": "WAITING" }
}
```

ERROR RESPONSES

400 not a PROCTORING exam or not open. **403** hierarchy mismatch. **404** no room assignment exists yet.

DATABASE

Exam, **ExamRoomAssignment**, **StudentAcademicDetail**

RELATED WORKFLOW

Proctoring Workflow (Section 7.6)

POST /faculty/exam-rooms/:roomId/assignments/:assignmentId/admit

PURPOSE

Admits one specific waiting student into the exam, creating their attempt record.

AUTHENTICATION / AUTHORIZATION

Required — **FACULTY** , and the room must belong to the calling faculty (checked server-side on every proctoring endpoint, not only this one).

BUSINESS RULES

The target assignment must currently be **WAITING** . Admission creates the student's **ExamAttempt** and transitions the assignment to **ADMITTED** .

SUCCESS RESPONSE — 200

```
{ "success": true, "message": "Student admitted", "attemptId": "665f..." }
```

ERROR RESPONSES

404 room/assignment not found, or not owned by the calling faculty. **409** assignment is not in **WAITING** status.

DATABASE

ExamRoom , **ExamRoomAssignment** , **ExamAttempt** (created)

PATCH /student/exams/attempts/:attemptId/answers/:questionId**PURPOSE**

Saves (upserts) the student's current answer to one question — called on every option selection or text change, not only on navigation.

AUTHENTICATION / AUTHORIZATION

Required — **STUDENT** , and the attempt must belong to the calling student.

REQUEST BODY

```
// MCQ: { "selectedOptionId": "opt-2" }  
// MSQ: { "selectedOptionIds": ["opt-1", "opt-3"] }  
// TYPING: { "answerText": "<p>Sanitized Tiptap HTML</p>" }
```

BUSINESS RULES

Attempt must be **IN_PROGRESS** . The field accepted depends on the question's type; a WRITTEN question is never saved through this endpoint — it uses the separate QR/photo-upload flow instead.

SUCCESS RESPONSE — 200

```
{ "success": true, "message": "Answer saved" }
```

ERROR RESPONSES

403 not the attempt owner. **409** attempt is not **IN_PROGRESS** .

DATABASE

ExamAnswer

POST /student/exams/attempts/:attemptId/submit**PURPOSE**

Finalizes the calling student's own attempt — manual submission path (the same finalization logic is also triggered automatically by the lifecycle sweep on timer expiry, and by the integrity monitor on repeated violations).

AUTHENTICATION / AUTHORIZATION

Required — **STUDENT**, owner of the attempt.

BUSINESS RULES

Attempt must be **IN_PROGRESS**. Objective questions (MCQ/MSQ) are scored immediately against stored correct-option sets; Written/Typing questions remain unscored until faculty evaluation. **submissionTrigger** is recorded (**MANUAL**, **TIMER_EXPIRY**, **INTEGRITY_AUTO_SUBMIT**, **FACULTY_REMOVED**, or **CONNECTION_LOSS**).

SUCCESS RESPONSE — 200

```
{
  "success": true,
  "message": "Exam submitted successfully",
  "data": { "totalScore": 34, "passed": true }
}
```

ERROR RESPONSES

403 not the attempt owner. **409** attempt already submitted/completed.

DATABASE

ExamAttempt, **ExamAnswer**, **Exam**

RELATED WORKFLOW

Student Examination Workflow (Section 7.4)

10.3 Standard Success Response Envelope

```
{
  "success": true,
  "message": "Human-readable description",
  "data": { } // present on most GET/detail endpoints; some mutation endpoints return
              // top-level fields instead (e.g. "examId", "attemptId") rather than a nested "data"
}
```

10.4 Standard Error Response Envelope

Errors are NestJS's default HTTP-exception JSON shape (not a custom wrapper):

```
{
  "statusCode": 400,
  "message": "Semester must be between 1 and 8 for this course",
  "error": "Bad Request"
}
```

`message` may be a single string (thrown business-rule exceptions) or an array of strings (a `ValidationPipe` rejection listing every failed field at once).

10.5 HTTP Status Codes Used

STATUS	MEANING	WHEN
200	OK	Successful GET, or a mutation that doesn't create a new resource.
201	Created	Successful POST that creates a resource (exam, batch, room, etc.).
400	Bad Request	DTO validation failure or a business-rule violation (<code>BadRequestException</code>).
401	Unauthorized	Missing/invalid/expired JWT.
403	Forbidden	Valid JWT, but the role or ownership check fails.
404	Not Found	Referenced entity does not exist, or does not belong to the calling user/faculty (ownership checks intentionally return 404 rather than 403 in several places, so as not to confirm the existence of a resource the caller has no right to know about).
409	Conflict	A state-machine precondition failed (e.g. admitting an already-admitted student, forming rooms twice).
500	Internal Server Error	An unexpected database or unhandled error.

11. API Access-Control Matrix

Every backend endpoint carries exactly one `@Roles()` value (or none, for the public/auth surfaces) — there is no endpoint accessible by more than one role. The matrix below groups the 98 endpoints by resource area rather than listing all 98 rows individually, since access is uniform within each group.

RESOURCE GROUP	SUPER ADMIN	FACULTY	STUDENT	PUBLIC
Authentication (login/logout/refresh/password)	✓	✓	✓	✓ (login/forgot/reset are unauthenticated by nature)
Academic hierarchy (batches/courses/departments/sections/subjects — admin lookups)	✓	X	X	X
Student management (CRUD, bulk upload, activity)	✓	X	X	X
Faculty management (CRUD, activity)	✓	X	X	X
Exam authoring (create/edit/delete/publish/questions)	✓	X	X	X
Exam room formation & admin room/attempt review	✓	X	X	X
Result publication & evaluator assignment	✓	X	X	X
Admin dashboard	✓	X	X	X
Subject management (own subjects)	X	✓	X	X
Proctoring room monitoring (own assigned rooms: list, detail, admit, reject, remove, mute, chat, LiveKit token)	X	✓	X	X
Evaluation queue (own assigned exams)	X	✓	X	X
Faculty own profile	X	✓	X	X
My Exams / start / answer / submit / result	X	X	✓ (own attempts only)	X
Lobby join & student-side proctoring (chat, LiveKit token)	X	X	✓ (own assignment only)	X
Recording upload signature/chunk/finalize; violation reporting; network probe	X	X	✓ (own attempt only)	X
Student own profile & subjects (read)	X	X	✓	X
Written-answer QR verify / upload signature / record page	X	X	X	✓ (QR token is the credential, not a logged-in session)

LiveKit webhook	X	X	X	✓ (HMAC-signed, LiveKit server only)
-----------------	---	---	---	--------------------------------------

OWNERSHIP BEYOND ROLE

Several groups above are further scoped by data ownership, not role alone: a faculty member can only act on a proctoring room whose `facultyId` matches their own id (`ownRoomOrThrow`), and a student can only act on their own attempt/assignment (`ownAttemptOrThrow`). This is enforced inside the service layer on every call, not only implied by the route structure — the role guard alone would not be sufficient to prevent one faculty member from viewing another's room.

12. Database Architecture

12.1 Overview

PROPERTY	DETAIL
Engine	MongoDB (document database), hosted on MongoDB Atlas
ODM	Mongoose 8.x via <code>@nestjs/mongoose</code>
Connection	<code>mongodb+srv://</code> Atlas SRV connection string, wired through <code>MongooseModule.forRootAsync</code>
Collections	31 — one Mongoose schema class per collection, no schema is shared/reused across collections
Naming convention	Mixed: several collections use an explicit <code>@Schema({collection: '...' })</code> name in <code>snake_case</code> (e.g. <code>exam_answers</code> , <code>exam_room_assignment</code>), while others fall back to Mongoose's default pluralized-lowercase class name (e.g. <code>batches</code> , <code>students</code>) — this is inconsistent but stable (not something to "fix" without a migration).
Primary keys	MongoDB's default <code>_id</code> (ObjectId) on every collection — no custom primary key is used anywhere.
Relationships	Modeled via <code>Types.ObjectId</code> reference fields (<code>ref: 'CollectionName'</code>) and resolved with either Mongoose <code>.populate()</code> or explicit batch-lookup queries in the service layer — there are no MongoDB-native joins beyond <code>populate</code> .
Soft deletion	Used selectively — <code>Exam.isDeleted</code> , <code>ExamQuestion.isDeleted</code> , <code>Student.isActive</code> / <code>Faculty.isActive</code> / <code>User.isActive</code> flag records inactive rather than removing them; there is no universal soft-delete convention across every collection.

KNOWN SCHEMA INCONSISTENCIES (VERIFIED IN SOURCE, NOT CORRECTED HERE)

- `StudentAcademicDetail` carries an index on a field named `admissionNumber` , which does not exist anywhere on that schema — a dead index, almost certainly left over from a field rename to `rollNumber` .
- The exam-room-assignment collection is named `exam_room_assignment` (missing an "n"), inconsistent with its sibling collections' correct spelling.
- Several collections declare an explicit `.index()` that duplicates a field already marked `unique: true` inline — redundant, not incorrect.
- `AuthActivityLog` defines its own local `AuthAction` enum instead of importing the shared enum module every other schema uses.

12.2 Collection Reference

Fields are condensed to name/type/notable constraint; full validation detail lives in the corresponding request DTOs (Section 10) where a field is user-supplied.

Academic collections

COLLECTION	KEY FIELDS	INDEXES
batches	batchName, startYear, endYear	—
courses	streamCode, streamName, courseCode (unique), courseName, level, duration, semesters	—
batchcourses	batchId → Batch, courseId → Course	—
departments	courseId → Course, deptCode, deptName	—
batchdepartments	batchCourseId → BatchCourse, courseId → Course, deptId → Department, totalSeats, sectionCapacity (default 50)	—
sections	batchId/courseId/departmentId → refs, sectionName, capacity, currentStrength	{batchId,courseId,departmentId,sectionName} unique
subjects	departmentId → Department, semester, subjectCode (unique), subjectName, credits, subjectType (enum), createdBy → Faculty, isActive	{departmentId,semester}

Identity & access collections

COLLECTION	KEY FIELDS	INDEXES
users	email (unique), password (bcrypt hash), role (enum), isActive, isEmailVerified, isFirstLogin, accessToken/refreshToken (current session tokens, persisted server-side), resetPasswordTokenHash, resetPasswordTokenExpiresAt, createdBy → User (self-ref)	{role,isActive}, {lastLogin}
authactivitylogs	userId → User, email, role, action (LOGIN/LOGOUT), ipAddress, userAgent	{userId,createdAt:-1}, {action,createdAt:-1}

Faculty collections

COLLECTION	KEY FIELDS	INDEXES
faculties	userId → User, facultyId (unique, e.g. FAC1024), personalDetailId / contactInformationId / employmentDetailId → refs (required), addressDetailId → ref, educationHistoryIds[] / workExperienceIds[] → ref arrays, isActive	{facultyId}, {userId}, {employmentDetailId}
facultypersonaldetails	firstName, lastName, gender, dateOfBirth, maritalStatus?, profilePhotoUrl?, nationality, religion?	—
facultycontactinformations	personalEmail, facultyEmail (unique), phoneNumber, alternatePhoneNumber?, emergencyContact? (name/relation/phone)	{facultyEmail}, {personalEmail}

facultyaddresses	currentAddress?, permanentAddress?, sameAsCurrent — all optional (self-serve profile completion)	—
facultyemploymentdetails	employeeId (unique), designation (enum), departmentId → Department, employmentType (enum), dateOfJoining, dateOfLeaving?, totalExperienceYears, highestQualification?, status (enum), remarks?	{employeeId}, {departmentId}, {designation,status}
facultyeducationhistories	facultyId → Faculty, level (enum), qualification, boardOrUniversity, institutionName, yearOfPassing, percentageOrCGPA, specialization?	{facultyId}
facultyworkexperiences	facultyId → Faculty, organization, role, department?, fromDate/toDate, experienceYears, isCurrent	{facultyId}, {facultyId,isCurrent}

Student collections

COLLECTION	KEY FIELDS	INDEXES
students	userId → User, studentId (unique, e.g. STU3021), personalDetailId / contactInformationId / academicDetailId → refs (required), addressDetailId → ref, educationHistoryId / parentDetailId? → optional refs, isActive	{studentId}, {userId}, {academicDetailId}
studentacademicdetails	rollNumber (unique), batchId/courseId/departmentId/sectionId → refs, currentSemester, admissionType (enum), status (enum)	{rollNumber}, {batchId,courseId,departmentId}, {sectionId} (+ a dead {admissionNumber} index — see 14.1)
studentpersonaldetails	firstName, lastName, gender, dateOfBirth, profilePhotoUrl? (self-serve), nationality, religion?	—
studentcontactinformations	personalEmail, studentEmail (unique), phoneNumbers, alternatePhoneNumber?, emergencyContact?	{studentEmail}, {personalEmail}
studentaddressdetails	currentAddress?, permanentAddress?, sameAsCurrent — not collected at admission time	—
studenteducationhistories	studentId → Student, level/qualification/boardOrUniversity (enums), institutionName, yearOfPassing, percentageOrCGPA, remarks?	{studentId}
studentparentdetails	studentId → Student, father? / mother? / guardian? — every nested field optional	{studentId}

Examination collections

COLLECTION	KEY FIELDS	INDEXES
exams	name, description?, mode (AUTO/PROCTORING), status (enum, default DRAFT), batchId/courseId/departmentId → refs, sectionIds[] → Section refs, semesters[] (numbers), subjectId →	{batchId,courseId,departmentId,subjectId}, {sectionIds}, {semesters}, {mode,status}, {startDate,endDate,startTime,endTime,durationMinutes}, {status}

	ref, durationMinutes, totalMarks, passingMarks, startDate, endDate, startTime?/endTime? (PROCTORING-only), examSections[] (embedded, own _id per entry), evaluatorFacultyIds[] → Faculty refs, securitySettings (embedded), isDeleted, createdBy/updatedBy → User	
exam_questions	examId → Exam, type (MCQ/MSQ/WRITTEN/TYPING), text, marks, order, examSectionId? (no formal ref — matches an embedded examSections entry's own _id), options[] (MCQ/MSQ), correctOptionIds[], isDeleted, createdBy/updatedBy	{examId,order}
exam_attempts	examId/studentId → refs, status (enum, default NOT_STARTED), questionOrder[] (per-student shuffle), optionOrder (map), roomId? → ExamRoom, startedAt/submittedAt, submissionTrigger (enum), mediaStatus (enum), objectiveScore/writtenScore/totalScore, passed, isFlagged, violations[] (type, occurredAt)	{examId,studentId} unique, {examId,status}, {roomId}, {examId,isFlagged}
exam_answers	attemptId/questionId → refs, selectedOptionId (MCQ), selectedOptionIds[] (MSQ), qrTokenHash/qrGeneratedAt/qrTokenExpiresAt/qrScannedAt (WRITTEN), pages[] {pageNumber,cloudinaryUrl,uploadedAt}, answerText (TYPING), firstViewedAt, isFinalized, marksAwarded, evaluatedBy → User, evaluatedAt, remarks	{attemptId,questionId} unique, {qrTokenHash} unique+sparse, {evaluatedBy,isFinalized}
exam_recordings	attemptId → ExamAttempt (unique), video/screen (each: status enum, chunks[] {sequence, cloudinaryAssetId, cloudinaryUrl, uploadedAt})	{attemptId} unique
exam_rooms	examId? (unset for a pooled/mixed room — see 14.1), facultyId → Faculty, startDateTime/endDateTime, durationMinutes, liveKitSessionId (unique), status (enum, default SCHEDULED)	{facultyId,startDateTime}, {startDateTime,endDateTime}, {examId}
exam_room_assignment (sic)	roomId/examId/studentId → refs, attemptId? (set on admission), status (enum, default WAITING), enteredWaitingRoomAt, admittedAt/By, removedAt/By/removalReason, disconnectedAt	{roomId,studentId} unique, {roomId,status}, {examId,studentId}
exam_room_chat_messages	roomId → ExamRoom, senderRole (enum), senderId → User, recipientType (INDIVIDUAL/BROADCAST_ROOM/BROADCAST_EXAM), recipientStudentId?, message, sentAt	{roomId,sentAt}

12.3 Data Lifecycle Notes

- An **ExamAnswer** row for every question is created up front when a student's attempt starts (WRITTEN questions get their QR-related fields populated later, on demand) — answers are never created lazily on first save.

- `ExamRoomAssignment.status` is a genuine state machine (`WAITING` → `ADMITTED/REJECTED` → `IN_PROGRESS` → `COMPLETED` , with `DISCONNECTED` as a non-terminal detour back to `WAITING`), not a free-text status field.
- Recording chunks accumulate in `exam_recordings` as they upload during the exam, then the stream's own `status` flips to a completed state on finalize — the recording document is never fully rewritten, only appended to.

13. Entity Relationship Diagrams

Given the number of collections (31), the schema is presented as two focused diagrams rather than one diagram attempting to show every relationship at once: academic/identity structure, and the examination cluster. Cardinality is shown as 1—1, 1—N, or N—N next to each connector.

13.1 Academic Structure & Identity

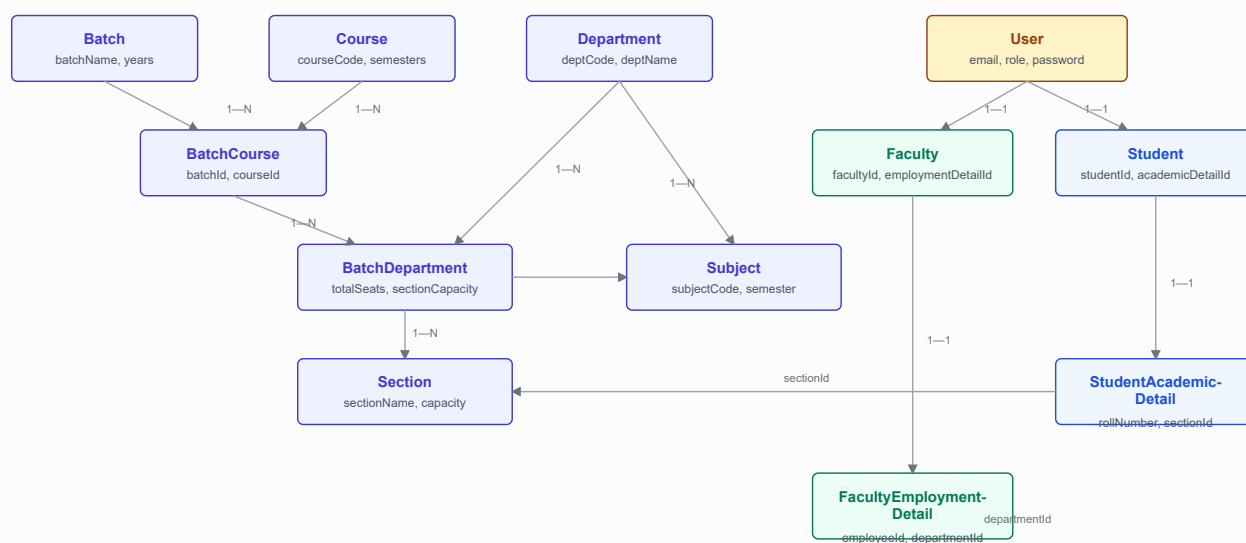


Figure 13.1 — Academic hierarchy and identity relationships. Personal/contact/address sub-documents for Student and Faculty (7 collections each, Section 12.2) are omitted here for readability — each is a simple 1—1 reference from Student/Faculty respectively.

13.2 Examination Cluster

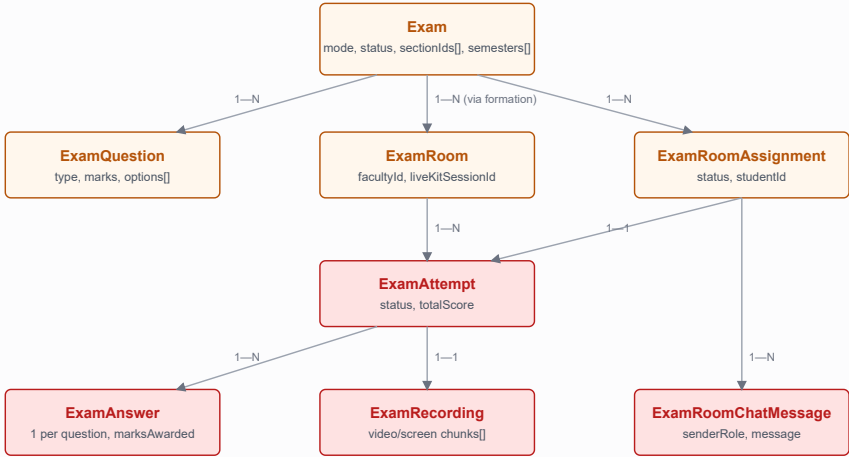


Figure 13.2 — Examination cluster. An **ExamRoom** does not always reference a single **Exam** directly (pooled/mixed rooms leave it unset — Section 17.2); per-student exam identity in that case lives on each **ExamRoomAssignment** / **ExamAttempt** row instead.

14. Data Flow Diagrams

14.1 Authentication Data Flow

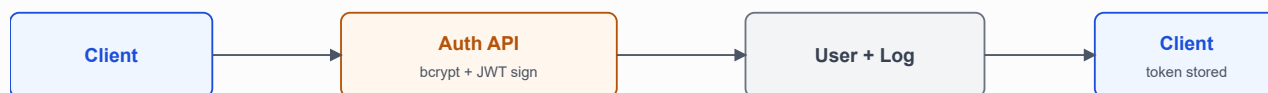


Figure 14.1 — Credentials → Auth API → User/AuthActivityLog write → token back to client. Full detail in Section 18.1.

14.2 Examination Data Flow

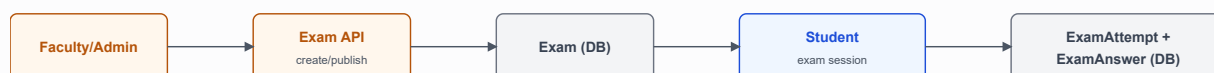


Figure 14.2 — Full detail in Section 7.3–7.5 (workflows) and Section 16 (Examination System).

14.3 Proctoring Data Flow (Monitoring Events)

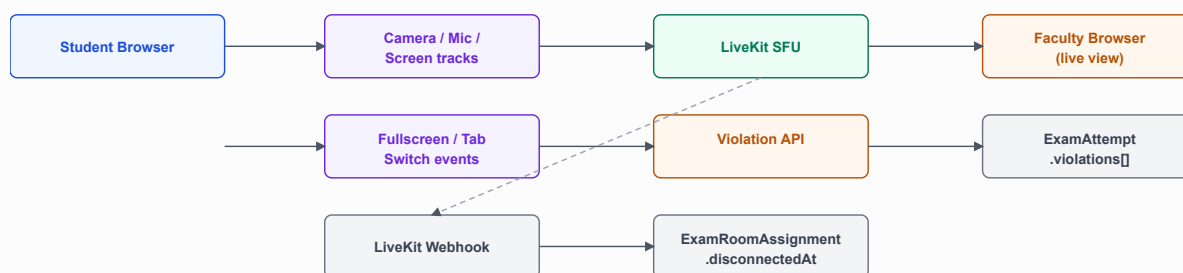


Figure 14.3 — Two independent event paths: media/fullscreen/tab-switch state flows through LiveKit and the violation-reporting API to the faculty's live view and the attempt's violation log; a LiveKit connection-loss event separately flows through the webhook into the room assignment's disconnect tracking. Full detail in Section 17 (Proctoring Architecture).

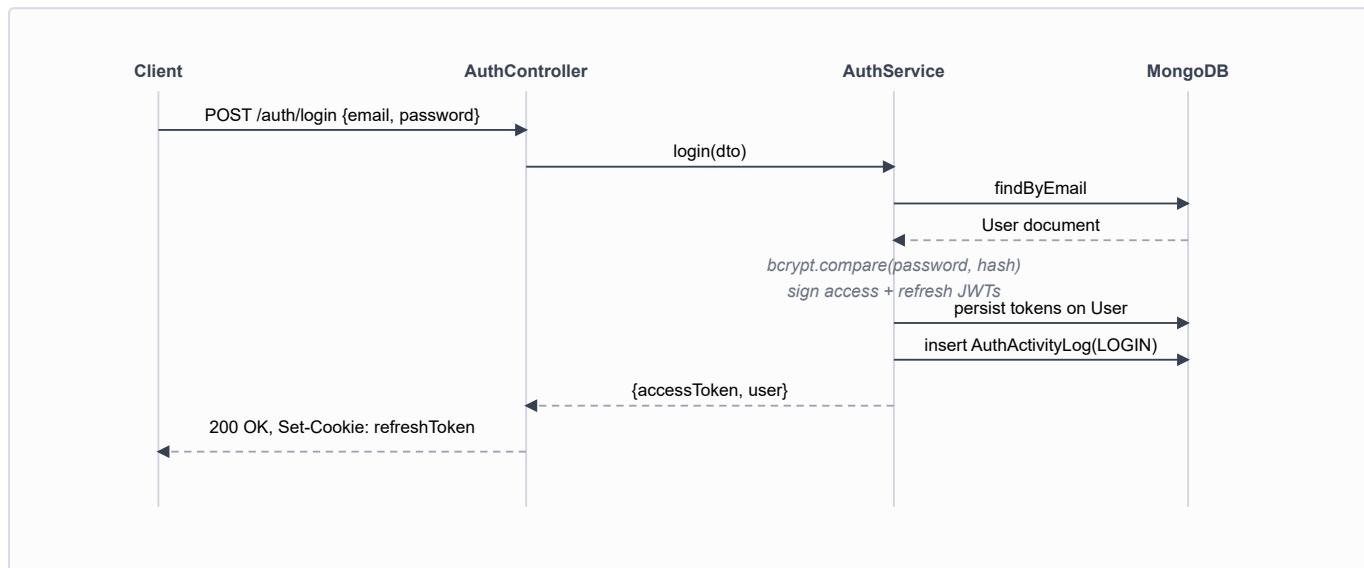
14.4 File Management Data Flow

See Figure 21.1 (Section 21.1) — the written-answer and recording upload flow is the system's only file-management data flow, and is diagrammed there in full rather than repeated here.

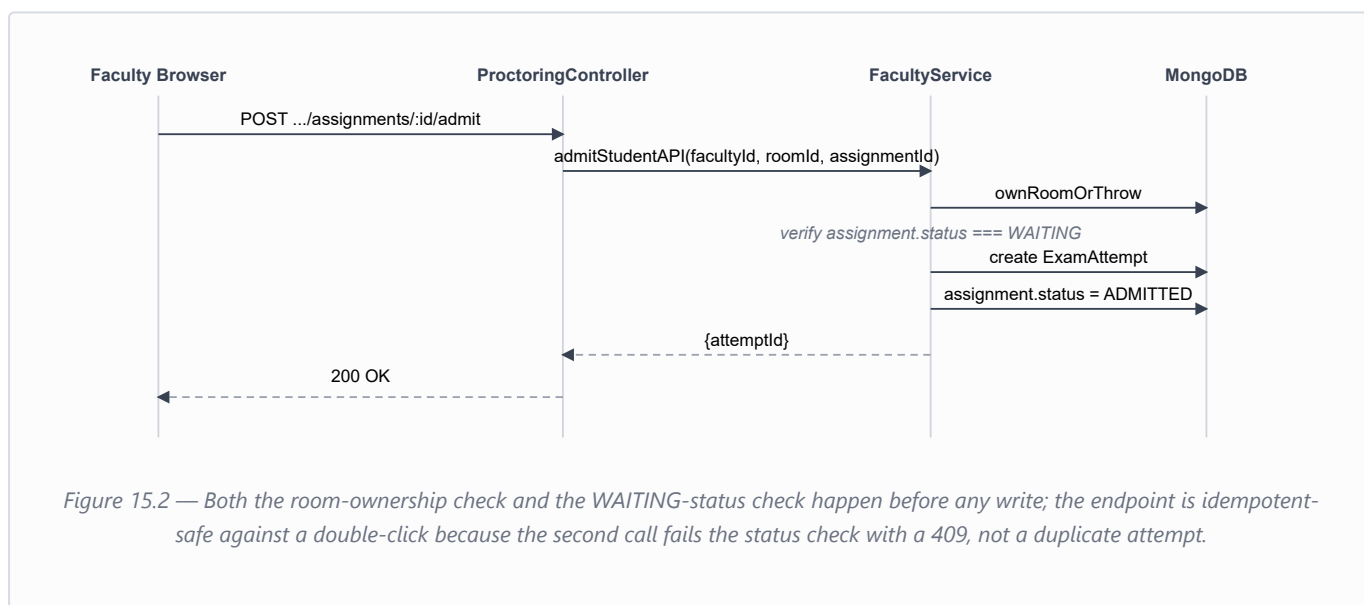
15. Sequence Diagrams

Only the participants that actually take part in each flow are shown.

15.1 Login



15.2 Faculty Admits a Waiting Student



15.3 Exam Submission (Manual and Automatic)

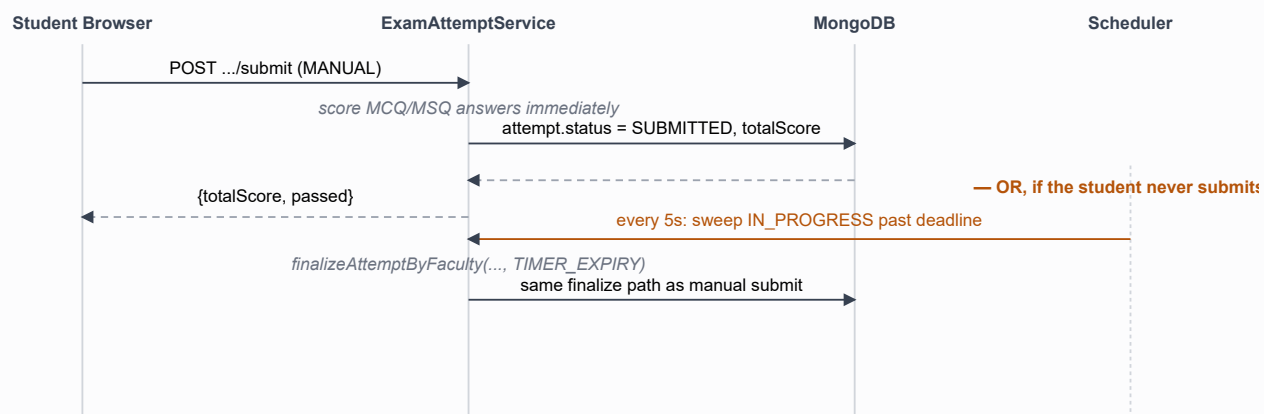


Figure 15.3 — Manual submission and the timer-expiry sweep share the exact same finalization code path, differing only in **submissionTrigger** — this is deliberate, so scoring logic is never duplicated between the two triggers.

16. Examination System

16.1 Exam Status State Machine

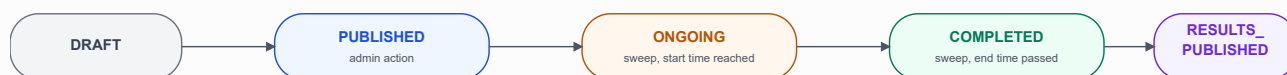


Figure 16.1 — Only the *DRAFT→PUBLISHED* and *COMPLETED→RESULTS_PUBLISHED* transitions are admin-triggered; the two middle transitions are entirely time-driven by the lifecycle sweep (Section 31.3), independent of anyone viewing the exam.

16.2 Exam Modes

MODE	SCHEDULE	ROOM/PROCTORING
AUTO	Date-only (<code>startDate</code> / <code>endDate</code>) — no time-of-day; a student may start any time within the full-day window and has until <code>durationMinutes</code> elapses from their own start.	None — no room formation, no live monitoring, taken directly after the environment check.
PROCTORING	Precise date + time-of-day (IST), required at creation.	Rooms must be formed by an admin; students join a waiting room and are admitted by their assigned faculty; live camera/screen monitoring for the whole session.

16.3 Question Types

TYPE	ANSWER CAPTURE	SCORING
MCQ	Single option selected from a fixed set (always 4 options in this implementation).	Automatic, at submission — exact match against the stored correct option.
MSQ	One or more options selected.	Automatic, at submission — exact-set match against the stored correct-option list.
WRITTEN	Student scans a per-question QR code with their phone and photographs each handwritten page; pages upload directly to Cloudinary and are "finalized" by the student before submission locks them in.	Manual — faculty evaluation queue.
TYPING	Rich-text answer authored in-browser (Tiptap), sanitized with DOMPurify before storage/display.	Manual — faculty evaluation queue.

16.4 Exam Sections

An exam may define named sections (`examSections` , e.g. "Section A", "Section B") purely as a question-grouping/navigation aid — this is unrelated to the academic `Section` (class/division) collection despite the shared name. A question optionally references one exam section via `examSectionId` ; questions with no section assigned are grouped as "Other Questions" in the student's navigation.

16.5 Multi-Section / Multi-Semester Targeting

An exam's eligibility is defined by `sectionIds[]` and `semesters[]` (both arrays, Section 25.1) evaluated as a cross-product AND: a student matches if their own academic section is any one of the exam's target sections *and* their own current semester is any one of the exam's target semesters. This lets one exam target, for example, both Section A and Section B of semester 3 in a single exam record instead of creating duplicate exams per section.

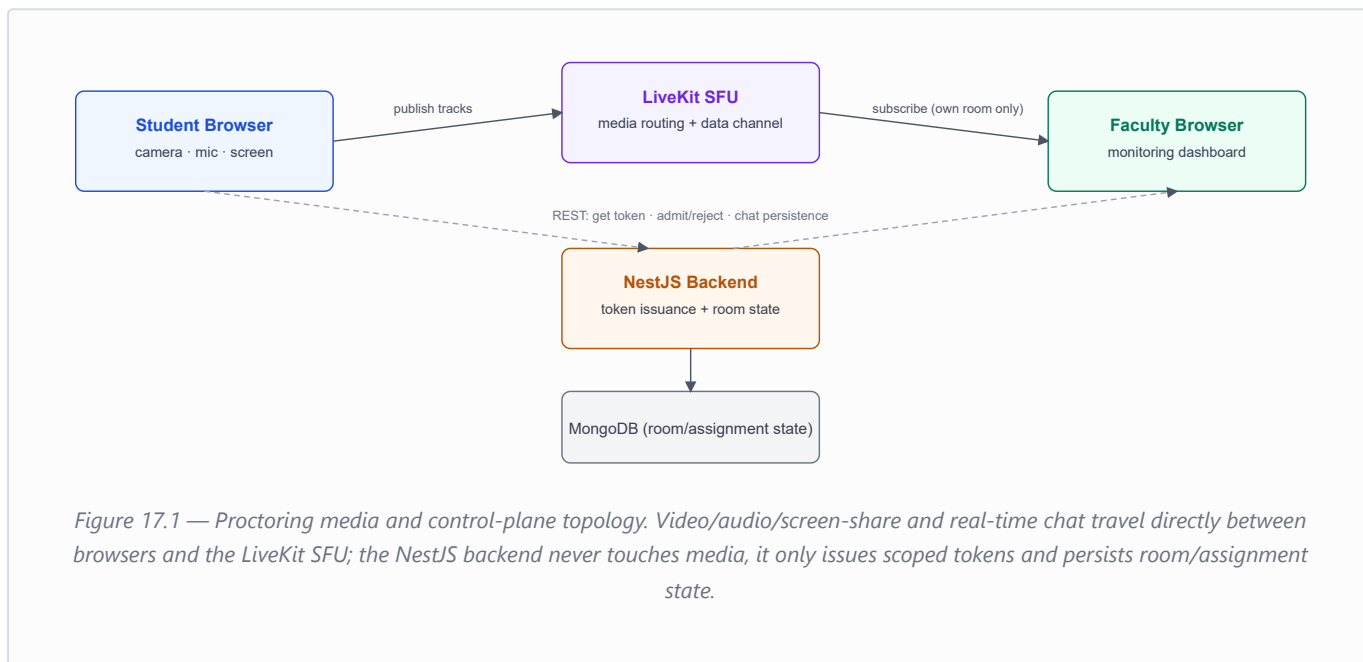
16.6 Security Settings (Per Exam)

SETTING	DEFAULT	EFFECT
<code>shuffleQuestions</code>	false	Per-student randomized question order (stored as <code>questionOrder</code> on the attempt, so it's stable across a resume).
<code>shuffleOptions</code>	false	Per-student randomized MCQ/MSQ option order.
<code>disableCopyPaste</code> / <code>disableRightClick</code>	true / true	Client-side deterrents during the exam.
<code>requireFullScreenThroughout</code>	true	Enforced by the fullscreen integrity check (Section 17.3).
<code>blockBackwardNavigation</code>	false	When enabled, a student cannot return to a previously-answered question.
<code>tabSwitchViolationThreshold</code> / <code>fullScreenExitViolationThreshold</code>	3 / 3	Violation count that triggers automatic submission.
<code>connectionLossGracePeriodMinutes</code>	2	Grace window before a disconnected PROCTORING student's attempt is finalized (Section 17.2).
<code>minTimePerQuestionSeconds</code> / <code>minTimePerExamMinutes</code>	0 (disabled) / 0 (disabled)	Minimum dwell time before advancing past a question, or before the whole exam can be submitted.

17. Proctoring Architecture

Live proctoring is the platform's most technically involved subsystem, combining a WebRTC SFU (LiveKit), a faculty-driven admission workflow, and a set of client-side integrity checks that feed back into the exam-attempt state machine.

17.1 Component Overview



17.2 Session Lifecycle

Room formation (pre-session)

Handled entirely by the admin-side `formExamRoomsAPI` before any student joins: matched students (computed from the exam's target hierarchy against each student's own academic detail) are partitioned into groups of at most 10, each group assigned an active faculty member round-robin, and an `ExamRoom` + one `ExamRoomAssignment` per student is created with status `WAITING`. Window-sibling PROCTORING exams (identical schedule) are pooled together so small leftover groups across exams share a room instead of each getting its own sparse room.

Waiting room and admission

A student's own action (not room formation) sets `enteredWaitingRoomAt` — the faculty dashboard only surfaces a student as visible/actionable once they have actually joined, not from the moment rooms were formed. Faculty then admit or reject each waiting student individually; admitting a student creates their exam attempt and transitions the assignment to `ADMITTED`.

Live session

Once admitted, the student's browser connects to LiveKit and publishes three tracks — camera, microphone, and (if granted) screen-share — restricting subscription permissions so only their own room's assigned faculty identity can ever receive them. The faculty's browser subscribes to every participant in the room and renders each student's camera and screen-share into per-student tiles, keyed by a `student-<id>` identity convention. Faculty can mute a student's microphone (a server-authoritative `RoomServiceClient` call, not just muting local playback), send/receive chat over LiveKit's data channel (mirrored to a REST call for persistence), and remove a student — which forcibly disconnects them and immediately finalizes their attempt.

Disconnect handling

A LiveKit disconnect is not treated as a removal. A scheduled sweep watches for assignments stuck in a disconnected state past a configurable grace period (`connectionLossGracePeriodMinutes` , per-exam security setting); within the grace period the student can simply rejoin and resume, and only past it does the system finalize their attempt.

17.3 Environment Check & Integrity Monitoring

CONDITION	WHAT IS CHECKED	WHAT HAPPENS WHEN TRIGGERED
Camera access	Pre-flight: <code>getUserMedia({video:true})</code> succeeds and a preview stream is obtained.	Blocks entry to the exam room until granted; retryable.
Microphone access	Pre-flight: <code>getUserMedia({audio:true})</code> .	Same as camera.
Screen-share	Pre-flight: <code>getDisplayMedia({video:true})</code> , entire-screen only (a single window/tab is rejected and the student is asked to retry).	Blocks entry until an entire-screen share is granted.
Multiple monitors	Pre-flight: the Window Management API (or a lighter <code>window.screen.isExtended</code> check) detects an extended display.	Flagged as a failed check the student must resolve (disconnect the extra monitor) before proceeding.
Fullscreen	Pre-flight entry, and continuously during the exam via the Fullscreen API.	Exiting fullscreen during the exam is logged as a violation.
Tab switch / window blur	Continuous, via visibility-change and blur listeners during the exam.	Logged as a violation.
Violation threshold	Per-exam configurable count of tab-switch/fullscreen-exit violations.	Once exceeded, the attempt is automatically submitted server-side (<code>SubmissionTrigger.INTEGRITY_AUTO_SUBMIT</code>) and the student is redirected out of the exam room.

Network quality	Pre-flight network probe against a minimum required throughput.	Warns the student before entry; does not currently block entry on its own.
-----------------	---	--

17.4 Faculty Monitoring Interface

The faculty proctoring dashboard (`ProctoringDashboardPage`) polls the room-detail endpoint every 3 seconds and additionally connects to LiveKit directly for live media and chat. Its state is deliberately gated on real activity: the full monitoring UI (waiting queue, video grid, chat panel) is only rendered once at least one student has actually joined the room, showing a simple "no student has joined yet" placeholder before that — avoiding a busy screen full of not-yet-relevant controls for a room nobody has entered.

18. Authentication & Authorization

Authentication answers "who is the user?" **Authorization** answers "what are they allowed to do?" XaminiyIQ keeps these as two distinct, sequential guard stages on every protected endpoint.

18.1 Authentication Flow

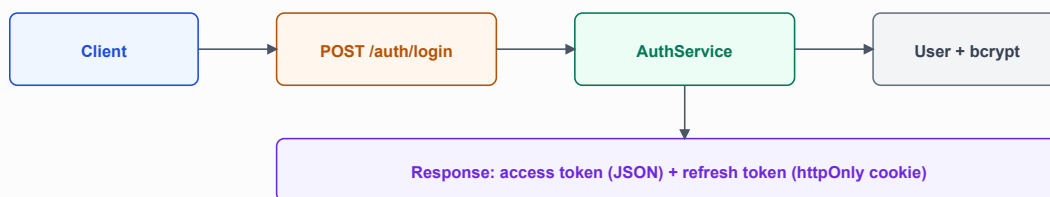


Figure 18.1 — Login. Both tokens are also persisted on the `User` document; logout clears them there, so a stolen access token stops working once the account owner logs out server-side, not only when it naturally expires.

18.2 Token Types

TOKEN	SIGNING SECRET	NOTES
Access token	JWT_SECRET_KEY	Short-lived (<code>JWT_EXPIRES_IN</code>); returned in the login response body; sent by the client as <code>Authorization: Bearer <token></code> on every subsequent request; also decoded client-side (not verified, just read) to proactively trigger a refresh before it expires.
Refresh token	JWT_REFRESH_SECRET_KEY	Longer-lived (<code>JWT_REFRESH_EXPIRY_IN</code>); set as an httpOnly, path-scoped (<code>/auth</code>) cookie — never readable by client-side JavaScript; on refresh, the exact stored token string on the <code>User</code> document must match, not only a valid signature, so revoking access by clearing the stored token invalidates a refresh token even before its JWT expiry.
Password-reset token	PASSWORD_RESET_JWT_SECRET_KEY	Fixed 10-minute expiry; its SHA-256 hash and expiry are separately stored on the user document and re-checked on reset, independent of the JWT's own expiry claim.
Written-answer QR token	QR_TOKEN_JWT_SECRET_KEY	Fixed 15-minute expiry; the sole credential for the unauthenticated <code>/public/written-answer/*</code> endpoints — the token itself, cross-checked against a stored hash on the <code>ExamAnswer</code> record, stands in for a logged-in session.

18.3 Session Refresh (Client-Side)

The frontend's Axios instance decodes the access token's expiry client-side and proactively calls `POST /auth/refresh-token` shortly before it lapses; any request issued while a refresh is already in flight is queued and replayed once the new token arrives, rather than each triggering its own refresh call. A reactive fallback also retries once on any 401 response. A failed refresh clears all local auth state and redirects to `/login`.

18.4 Authorization — Role-Based Access Control

Exactly three roles exist (`ADMIN` , `FACULTY` , `STUDENT`). Authorization is enforced at three independent layers, deliberately redundant rather than relying on any single one:

1. **Backend guard** — `RoleGuard` reads a `@Roles()` decorator per controller method and compares it against the authenticated `request.user.role`. This is the only layer that cannot be bypassed by a malicious client.
2. **Backend ownership check** — inside the service layer, several endpoints additionally verify the requested resource belongs to the caller (a faculty's own room, a student's own attempt) — role alone is not sufficient for these.
3. **Frontend route guard** — a `RoleGuard` component wraps every dashboard route, redirecting to `/unauthorized` on mismatch; this is a UX convenience, not a security boundary, since the backend enforces the same rule independently.

18.5 Password Security

- Hashed with bcrypt, 12 salt rounds; plaintext is never persisted or logged.
- Strength policy: minimum 8 characters, at least one lowercase letter, one uppercase letter, one digit (no symbol requirement is enforced).
- `User.isFirstLogin` forces a mandatory password change flow on a newly created account's first login.
- Forgot-password is intentionally unavailable for admin accounts (an explicit business rule in `auth.service.ts`) — an admin's password can only be reset by another mechanism outside this API (e.g. direct database access or a future admin-to-admin reset feature).

19. Security Architecture

19.1 Implemented Controls

CONTROL	DETAIL
Authentication	JWT access + httpOnly refresh cookie (Section 18).
Authorization	Role-based, enforced per-endpoint server-side (Section 18.4).
Password hashing	bcrypt, 12 rounds.
Input validation	Global <code>ValidationPipe</code> with <code>whitelist</code> and <code>forbidNonWhitelisted</code> — any request property not declared on the target DTO causes the whole request to be rejected, not silently dropped.

XSS mitigation	Typing-question answers are authored as HTML (via Tiptap) and sanitized with DOMPurify on the client before every render — not only at submission time, at every place the HTML is displayed (student's own editor preview, faculty evaluation queue, admin answer review).
CORS	Restricted to exactly two configured origins (<code>FRONT_END_BASE_URL_1/2</code>), credentialed.
Cookie security	Refresh-token cookie is httpOnly always, and additionally <code>secure: true</code> + <code>sameSite: 'none'</code> in production.
Webhook authenticity	The LiveKit webhook is verified via HMAC signature over the raw request body (not JSON-parsed first) using LiveKit's own API secret, not a session token.
Proctoring media privacy	A student's published camera/mic/screen tracks are restricted, via LiveKit's per-participant subscription permissions, to be visible only to their own room's assigned faculty identity — not to other students in the same room, and not to faculty of a different room.
Signed, time-limited uploads	File uploads to Cloudinary (written-answer photos, recording chunks) use short-lived, server-signed upload credentials scoped to a specific folder/public-id — the client never receives Cloudinary's actual API secret.

19.2 Controls Not Implemented

CONFIRMED ABSENT — NOT CLAIMED ELSEWHERE IN THIS DOCUMENT

- **No rate limiting** on any endpoint, including `/auth/login` and `/auth/forgot-password` — no `@nestjs/throttler` or equivalent is present.
- **No HTTP security-header middleware** (no Helmet or equivalent) — headers like `Content-Security-Policy`, `X-Frame-Options`, or `Strict-Transport-Security` are not explicitly set by the application.
- **No response compression middleware** configured.
- **No file-content validation** on the backend for Cloudinary uploads beyond what the signed upload parameters (folder, public_id) constrain — file type/size validation for the direct-to-Cloudinary path happens client-side, not re-verified by the NestJS server after the fact.

19.3 Secrets Management

All secrets (JWT signing keys — four distinct ones, database credentials, Cloudinary and LiveKit API credentials, the seeded initial admin password) are read exclusively from environment variables via a single `ConfigService` access point (Section 23) — no secret is hardcoded in source. No secret-manager service (e.g. AWS Secrets Manager, HashiCorp Vault) is integrated; environment variables are the sole mechanism.

20. External Services & Integrations

Exactly three external services are integrated. No AWS service, payment gateway, email/SMS provider, or CRM/ticketing integration exists anywhere in the codebase — confirmed by a full-repository grep for common provider SDKs and package names, with zero matches.

20.1 MongoDB Atlas

Purpose	Primary datastore for all 31 collections.
Integration method	<code>mongodb+srv://</code> connection string via Mongoose, wired through <code>MongooseModule.forRootAsync</code> .
Authentication	Username/password embedded in the connection string (read from <code>MONGODB_URI</code>).
Failure handling	Connection-level <code>connected</code> / <code>open</code> / <code>error</code> event listeners log to console; there is no automatic reconnect/retry logic layered on top of Mongoose's own built-in reconnection behavior.

20.2 Cloudinary

Purpose	Stores written-answer page photographs and exam-session recording chunks (video + screen-share).
Integration method	Server-signed, time-limited direct-to-Cloudinary uploads — the client uploads the file bytes straight to Cloudinary using a signature the backend generates (<code>cloudinary.utils.api_sign_request</code> over <code>{timestamp, public_id, folder}</code>); the backend never buffers file content itself for this path.
APIs used	Signed-upload signature generation, asset destroy-by-public-id, and a buffered server-side upload method (<code>uploadImage</code>) that exists in the service but is not currently reachable from any active endpoint — the one controller that would call it (student profile-photo upload) has its file-upload wiring commented out.
Data exchanged	Image/video binary content (client → Cloudinary directly); the backend only exchanges small JSON metadata (signature, resulting URL/asset id) with Cloudinary's API.
Security considerations	The Cloudinary API secret never leaves the backend; each signature is scoped to a specific folder and public_id, limiting what the holder of a signed upload URL can actually overwrite.

20.3 LiveKit

Purpose	Real-time camera, microphone, and screen-share streaming, plus data-channel signaling, for PROCTORING-mode exams.
---------	---

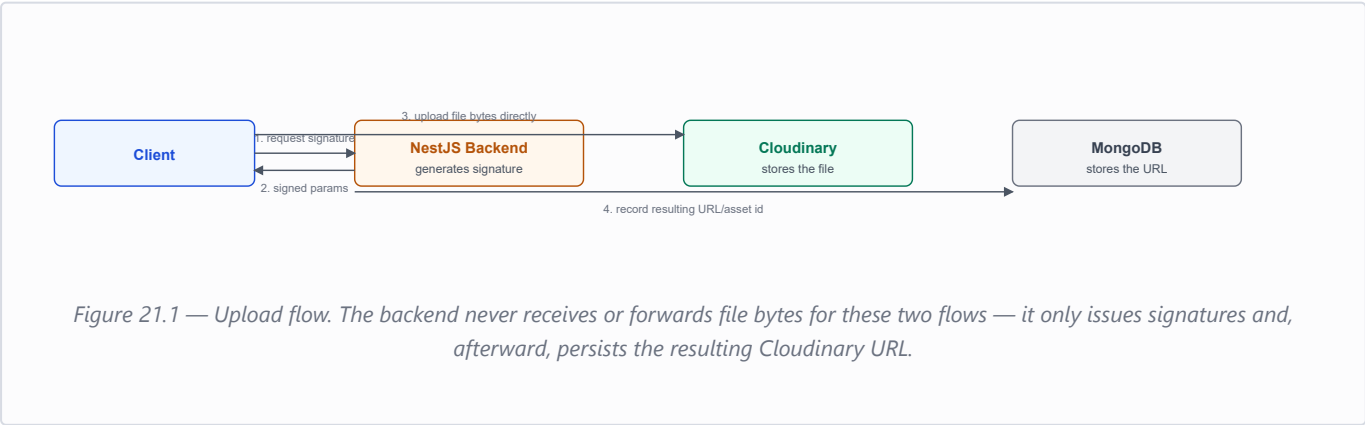
Integration method	Server-issued, room-scoped <code>AccessToken</code> s (via <code>livekit-server-sdk</code>) for both students and faculty; an HMAC-verified webhook receiver consumes LiveKit's own <code>participant_left</code> / <code>participant_joined</code> events.
APIs used	<code>AccessToken</code> generation with room-level grants (<code>roomJoin</code> / <code>canPublish</code> / <code>canSubscribe</code> / <code>canPublishData</code>); <code>RoomServiceClient.removeParticipant()</code> (faculty-forced disconnect); <code>RoomServiceClient.getParticipant()</code> + <code>mutePublishedTrack()</code> (server-authoritative mic mute); <code>WebhookReceiver</code> for connection-state events.
Authentication mechanism	Each participant's own signed JWT-style access token (LiveKit's token format), scoped to exactly one room (<code>liveKitSessionId</code>).
Data exchanged	WebRTC media (camera/mic/screen-share) flows directly between browsers and the LiveKit SFU — never through the NestJS backend. In-session chat is <i>not</i> carried over LiveKit's data channel for persistence — it is mirrored to a plain REST endpoint backed by the <code>exam_room_chat_messages</code> collection, with the LiveKit data channel used only for the live, in-the-moment delivery.
Failure handling	Disconnects are handled by a dedicated grace-period sweep (Section 24.3), not by LiveKit itself — the application treats a disconnect as recoverable, not immediately terminal.

20.4 Confirmed Absent

No AWS SDK, AWS Cognito, Amazon S3, Amazon Bedrock, Amazon Transcribe/Polly, or AWS Verified Permissions/Cedar is used anywhere in either repository. No email provider (SES, SendGrid, Mailgun, SMTP, or otherwise) or SMS provider is integrated — the password-reset link is currently only written to the server's console log with an explicit `// TODO: integrate email service` comment in the source. No payment gateway (Stripe, Razorpay, PayPal) exists. Redis (`ioredis`) is a declared package dependency with config-service accessor methods defined, but no `RedisModule` is ever imported or used anywhere in the running application — it is a dead dependency, not an active integration.

21. File & Storage Architecture

21.1 Upload Flow (Written-Answer Pages & Recording Chunks)



21.2 Storage by Content Type

CONTENT	STORAGE	NOTES
Written-answer page photographs	Cloudinary, folder per answer	Uploaded from a phone browser via the QR-scanned flow; each page is a separate signed upload; deletable individually before finalization.
Exam-session recording chunks (video + screen)	Cloudinary	Uploaded in sequence during the exam via a signature-per-chunk flow; chunk metadata (sequence, URL, upload time) accumulates on the <code>ExamRecording</code> document.
Profile photos (student/faculty)	Schema field exists (<code>profilePhotoUrl</code>)	The upload path is currently disabled in code (commented-out multer/file wiring on the create-student controller) — the field can be populated via the self-serve profile-update endpoint accepting a URL string, but there is no active server-side upload flow for it today.

21.3 Deletion

A written-answer page can be deleted individually before its parent answer is finalized (removed both from the Cloudinary asset and the `pages[]` array) — `CloudinaryService.destroyByPublicId`. No automatic cleanup job removes orphaned Cloudinary assets if a delete request fails partway; this is a manual/operational concern, not automated.

22. Deployment Architecture

22.1 Environments

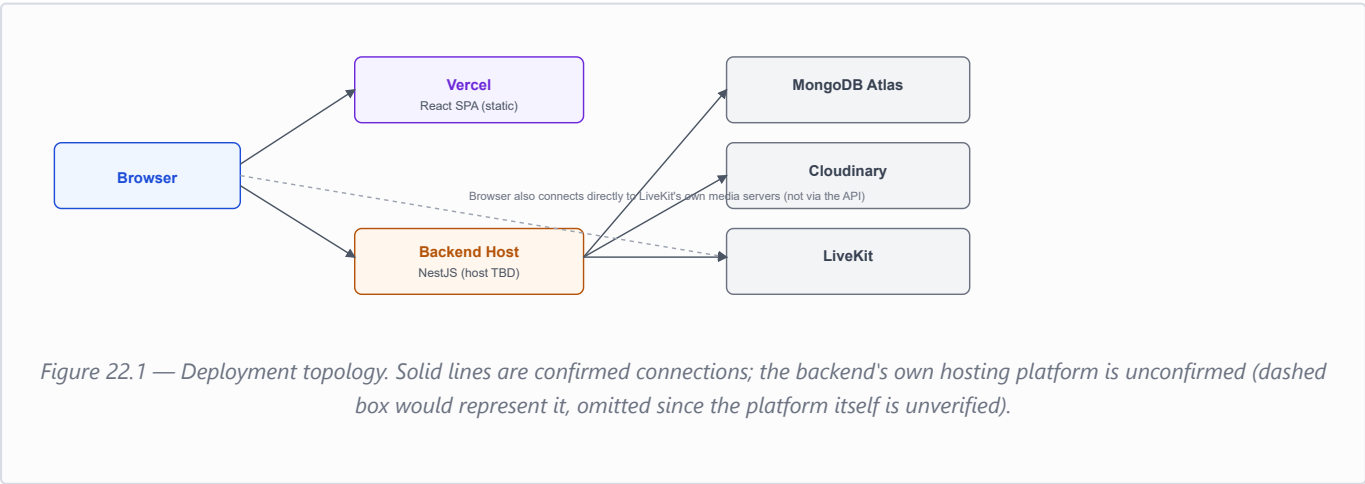
ENVIRONMENT	DETAIL
Development	Local — backend via <code>npm run start:dev</code> (Nest CLI watch mode) on port 8004 (default) / configured <code>PORT</code> ; frontend via <code>npm run dev</code> (Vite), default 5173.
Staging	Not present — no separate staging environment or configuration is defined in either repository.
Production	Frontend confirmed deployed to Vercel (a committed <code>vercel.json</code> with an SPA rewrite rule and the backend's CORS configuration references a <code>*.vercel.app</code> origin). Backend hosting is to be verified — no deployment manifest exists in the server repository; a Render.com hostname appears only in a commented-out line of the client's <code>.env.example</code> , which is circumstantial evidence, not a confirmed deployment configuration.

22.2 Build & Start Process

LAYER	BUILD	RUN
Backend	<code>nest build</code>	<code>node dist/main</code>
Frontend	<code>tsc -b && vite build</code>	Static output served by Vercel

Neither repository contains a Dockerfile, docker-compose configuration, or GitHub Actions (or any other CI) workflow — builds and deployments are not automated in-repo at the time of writing.

22.3 Deployment Topology



23. Environment Configuration

Every value below is a variable **name** read by the application; no actual secret value from either repository's `.env` is reproduced here.

23.1 Backend (`XaminityIQ-Server/.env`)

```
PORT=<port, e.g. 8004>
NODE_ENV=<development|production>
MONGODB_URI=<configured securely>

FRONT_END_BASE_URL_1=<primary allowed CORS origin>
FRONT_END_BASE_URL_2=<secondary allowed CORS origin>

JWT_SECRET_KEY=<configured securely>
JWT_EXPIRES_IN=<e.g. 15m>
JWT_REFRESH_SECRET_KEY=<configured securely>
JWT_REFRESH_EXPIRY_IN=<e.g. 7d>
PASSWORD_RESET_JWT_SECRET_KEY=<configured securely>
QR_TOKEN_JWT_SECRET_KEY=<configured securely>

INITIAL_ADMIN_EMAIL=<seed-only, first admin account>
INITIAL_ADMIN_PASSWORD=<configured securely>

CLOUDINARY_CLOUD_NAME=<account name>
CLOUDINARY_API_KEY=<configured securely>
CLOUDINARY_API_SECRET=<configured securely>

LIVEKIT_API_KEY=<configured securely>
LIVEKIT_API_SECRET=<configured securely>
LIVEKIT_URL=<LiveKit server URL>
```

DECLARED BUT UNUSED

`REDIS_HOST` , `REDIS_PORT` , `REDIS_PASSWORD` have accessor methods on `ConfigService` but are never consumed by any module — see Section 20.4.

23.2 Frontend (`XaminityIQ-Client/.env`)

```
VITE_BACKEND_BASE_URL=<API base URL, e.g. http://localhost:8004>
```

This is the only environment variable referenced anywhere in the client codebase.

README DISCREPANCY

Both repositories' `README.md` files document environment variable names that do not match what the application code actually reads (e.g. the server README instructs setting a frontend-style `VITE_API_BASE_URL` in the backend's own

`.env` ; the client README references `VITE_API_BASE_URL` / `VITE_BACKEND_URL` instead of the real `VITE_BACKEND_BASE_URL`). Use this section, not either README, as the authoritative environment-variable reference.

24. Logging & Monitoring

24.1 Application Logging

Logging is NestJS's built-in `Logger` plus direct `console.log` / `console.error` calls — there is no structured/JSON logging, no external log-aggregation service, and no dedicated APM/monitoring product integrated. What is logged: MongoDB connection lifecycle events, LiveKit connection failures, and the scheduled sweeps' per-item failures (each sweep item is wrapped in its own try/catch so one student's failure doesn't abort the sweep for everyone else).

24.2 Audit / Activity Logging

One genuine, persisted audit trail exists: `AuthActivityLog` records every login/logout with the acting user, role, timestamp, IP address, and user-agent — queryable per-student and per-faculty via the "activity" endpoints (`GET /admin/students/:id/activity` , `GET /admin/faculty/:id/activity`). There is no equivalent audit trail for other sensitive actions (e.g. exam edits, student removal from a proctoring session, evaluation score changes) — those are visible only through their effect on the relevant record, not as a separate change log.

24.3 Scheduled Background Processes

JOB	INTERVAL	RESPONSIBILITY
ExamLifecycleSchedulerService	Every 5 seconds	Force-finalizes overdue IN_PROGRESS attempts; force-completes attempts stuck in SUBMITTED for > 15 minutes; advances exam status PUBLISHED→ONGOING→COMPLETED based on the configured schedule.
LiveKitDisconnectSweepService	Every 60 seconds	Flips a room assignment to DISCONNECTED once its configured grace period elapses without reconnect, or force-finalizes the attempt if the exam's own window has already closed by then.

24.4 What Must Not Be Logged

Passwords (plaintext or hashed), full JWT token strings, and Cloudinary/LiveKit API secrets are never written to any log statement in the codebase — verified by inspection of every `console.log` / `Logger` call site in the auth and integration services. Personally identifiable student/faculty contact information (email, phone) is not logged outside the `AuthActivityLog` collection's own structured fields.

25. Technical Challenges & Solutions

The challenges below are real defects and design problems encountered during development of this project, kept here with their actual root cause and fix rather than a generic "lessons learned" narrative.

25.1 MongoDB Rejecting a Compound Index Across Two Array Fields

PROBLEM

Migrating an exam's academic targeting from single `sectionId` / `semester` fields to arrays (`sectionIds` / `semesters` , to support multi-section and multi-semester exams) caused every exam-creation request to fail with a database error: `cannot index parallel arrays [semesters] [sectionIds]` .

ROOT CAUSE

The migrated schema kept a single compound index — `{ batchId, courseId, departmentId, sectionIds, semesters, subjectId }` — that now had two multikey (array) fields in the same compound index. MongoDB disallows this outright: indexing two arrays together would require indexing their Cartesian product, which it refuses to build.

INVESTIGATION

The error only surfaced on the first document insert with real data in both arrays, not at server startup — Mongoose's default index sync only ever adds indexes it finds missing, it does not drop or repair one that changed shape, so the broken index had already been silently created against the live MongoDB Atlas cluster before the bug was even visible.

SOLUTION

The single compound index was split into three: one scalar-only compound index (`batchId, courseId, departmentId, subjectId`) plus two separate single-field multikey indexes (`sectionIds` , `semesters`). Because the bad index already existed in the database, a one-off script connected directly to the Atlas cluster to drop it and the stale pre-migration index, then create the corrected set.

RESULT

Exam creation with multi-section/multi-semester targeting works correctly, and the codebase now carries an explicit comment at the index declaration explaining why the fields are split, preventing the same mistake on a future schema change.

25.2 Stale Status Read Causing a False "Exam Must Be PUBLISHED" Rejection

PROBLEM

An admin reported being unable to form exam rooms for an exam they could see was `PUBLISHED` on screen — the request failed with *"Exam must be PUBLISHED before forming rooms."*

ROOT CAUSE

Two compounding issues. First, a scheduled sweep (`ExamLifecycleSchedulerService`) automatically advances exam status from `PUBLISHED` to `ONGOING` the moment the exam's configured start time passes — independent of any user action. Second, the client's "Form Exam Rooms" button visibility only checked `status !== DRAFT`, not `status === PUBLISHED` specifically, so the button stayed visible (and clickable) even after the exam had already advanced past `PUBLISHED`, and the page never refetched the exam in the background to notice.

INVESTIGATION

The server-side guard (`exam.status !== ExamStatus.PUBLISHED`) was correct and intentional — room formation genuinely should not be allowed once an exam is already running. The defect was a mismatch between what the client displayed and the server's authoritative, time-driven state.

SOLUTION

The client's room-formation gate was tightened to match the server exactly (`status === PUBLISHED` only), and the exam-detail page now polls the exam record every 5 seconds while it is `PUBLISHED`, so the UI self-corrects — the button disappears — the instant the sweep advances the exam, instead of failing silently on click.

RESULT

The admin now sees an accurate, live status and the corresponding action always matches what the server will actually accept.

25.3 Schema Evolution Without Data Backfill

PROBLEM

The Super Admin exam-detail page crashed in production with `Cannot read properties of undefined (reading 'find')` when opening certain exams.

ROOT CAUSE

An earlier change added an `examSections` array field to the `Exam` schema. Mongoose schema defaults (e.g. `default: []`) only apply when a document is created — they are never retroactively applied to documents that already existed before the schema change. Exams created before that change genuinely had no `examSections` field in the database, and the frontend code assumed the field would always be an array.

SOLUTION

The affected component was changed to derive a safe local value (`exam?.examSections || []`) instead of assuming the field's presence, with a comment recording why the fallback is necessary. The same "unguarded access to a field that predates its own schema addition" pattern was then proactively checked for in the two other places the exam room/mode change had touched.

RESULT

The immediate crash was fixed, and the incident established a standing rule for this codebase: any new optional field added to an existing schema must be read defensively everywhere it's consumed, not just where it was first introduced.

25.4 A UI Loading Flag Shared Across Unrelated List Rows

PROBLEM

On the faculty proctoring dashboard, clicking "Admit" for one waiting student made *every* Admit button in the Waiting Queue show a loading spinner and become disabled — visually indistinguishable from admitting every student at once.

ROOT CAUSE

The admit action's `isLoading` flag came from a single instance of the `useAdmitStudentMutation()` hook, called once per page rather than once per row. Every row in the waiting-queue list read that same shared boolean for its own button's `loading` prop.

INVESTIGATION

The actual mutation call was verified to be correctly scoped — it always passed the specific clicked student's `assignmentId`, and the server cross-checks that assignment against both the room and the calling faculty before admitting anyone. This was purely a client-side state-scoping bug, not a data-correctness or authorization bug, which mattered for prioritizing the fix (no server change, no audit of past admissions needed) but did not make the reported symptom any less confusing to whoever was using it.

SOLUTION

Replaced the single shared `isAdmitting` boolean with a per-assignment `admittingId` state value, so each row's loading state is independent, and disabled the other rows' buttons while one admission is in flight (rather than leaving them silently clickable) to prevent a double-submission race.

RESULT

Admitting a student now visibly affects only that student's row.

25.5 Timezone-Naive Date Construction in Exam Scheduling

PROBLEM

Exam start/end windows computed from separately-stored date and time-of-day values were opening roughly 5 hours 30 minutes earlier than an IST-thinking admin expected.

ROOT CAUSE

`new Date("2026-08-27T09:00:00")`, with no explicit UTC offset, is interpreted in the server process's own local timezone — typically UTC on a cloud host — not IST, the timezone the entire application's business logic assumes.

SOLUTION

A single shared helper, `combineDateTimeIST(date, time)`, was introduced to construct every exam-related date/time value with an explicit `+05:30` offset, and every call site that previously built a `Date` directly was migrated to use it. When AUTO-mode exams were later changed to be date-only (no time-of-day collected at all — see 31.6), the same helper was reused with sentinel times (`"00:00"` / `"23:59"`) rather than introducing a second, parallel date-construction path.

RESULT

Every exam-window calculation in the system — creation-time validation, the lifecycle sweep, and the student-facing "is this exam open right now" check — shares one IST-aware code path.

25.6 Redesigning a Required Field to Be Mode-Conditional Without Breaking Existing Data

PROBLEM

AUTO-mode (self-paced) exams needed to become date-only, dropping the previously mandatory start/end time-of-day fields — but PROCTORING exams still needed precise times for room scheduling, and several backend code paths (the student "can I start this exam" gate, a LiveKit disconnect sweep) unconditionally combined a date with a time string, which would break if that string became `undefined`.

TECHNICAL REASONING

Rather than making time-of-day fields required-but-nullable and scattering null checks, the schema fields were made genuinely optional (`startTime?` / `endTime?`), validation was branched by exam mode inside the existing hierarchy/schedule validator, and every remaining consumer of those fields was updated to fall back to a full-day sentinel window for AUTO exams — keeping exactly one date-construction code path (31.5) instead of introducing a second one for the date-only case.

RESULT

AUTO exam creation no longer collects or stores time-of-day data, PROCTORING exams are unaffected, and no existing PROCTORING exam's scheduling behavior changed.

26. Architecture & Design Decisions

Each entry states the decision actually made, the alternatives that were realistically available, why the chosen approach won, and the trade-off accepted.

DECISION	ALTERNATIVES CONSIDERED	REASON FOR SELECTION	TRADE-OFF ACCEPTED
NestJS for the backend	Express/Koa with a hand-rolled structure; a serverless function-per-endpoint approach	NestJS's module/controller/service DI structure maps directly onto the project's <code>api → service → repository → schema</code> layering, and its decorator-based guards/pipes give authentication, authorization, and validation as first-class, declarative concerns rather than hand-wired middleware.	More boilerplate per endpoint (a controller + request DTO + response DTO file per route) than a minimal Express setup would require.
MongoDB / Mongoose over a relational database	PostgreSQL with Prisma or TypeORM	The domain has several naturally document-shaped, nested structures (an exam's security settings, a question's options, a student's academic detail) that would otherwise need extra join tables, and the academic hierarchy's relationships (batch↔course↔department↔section) are shallow and few, not the deep relational graph a relational database is chosen for.	Referential integrity (e.g. a dangling <code>facultyId</code> reference after a faculty is deleted) is the application's responsibility, not the database's — every deletion path had to be reasoned about explicitly.
Layered service/repository split	Fat controllers calling Mongoose models directly	Keeps controllers as thin request/response translators, services as the sole home of business rules (e.g. exam eligibility, room-formation partitioning), and repositories as the only layer that knows Mongoose query syntax — a service can be reasoned about without knowing it's backed by MongoDB.	More files and indirection for simple CRUD than calling the model directly would need.
JWT access + httpOnly-cookie refresh token	Session cookies only; JWT stored in localStorage with no refresh mechanism	A short-lived access token limits the exposure window if it leaks (it's readable by JS, since the client needs it for the <code>Authorization</code> header), while the long-lived refresh token never touches JavaScript at all, since it's httpOnly.	Requires an interceptor pipeline on the client (proactive + reactive refresh, request queueing during refresh) rather than a single static token.
Role-based access control with exactly three fixed roles	A general permission/claim system (e.g. individual permission flags per user)	The domain has three genuinely distinct actors with non-overlapping responsibilities and no requirement for finer-grained permissions within a role — a fixed enum plus a <code>@Roles()</code> guard is simpler to reason about and audit than a permissions table, at the actual scale of this system.	Adding a fourth role or a partial-permission variant of an existing role (e.g. a "read-only admin") would require a new enum value and a guard-by-guard review, not a data-driven permission change.
LiveKit (self-hosted-capable WebRTC SFU) for proctoring	A custom WebRTC mesh; a recording-only (non-live) proctoring approach	A student's camera, microphone, and screen-share all need to reach one faculty member with low latency and without routing media through the application server; an SFU is the standard architecture for this, and LiveKit's client/server	Introduces a third runtime dependency (the LiveKit server) and a real-time failure mode (disconnects) that the application has to explicitly detect and

		SDKs directly map onto NestJS token issuance and React client integration.	recover from — see the disconnect-sweep grace-period mechanism in Section 17.
Cloudinary for file storage, via signed client-side uploads	Uploading files through the NestJS backend to storage; a self-hosted object store	Signed upload credentials let the client upload large files (recording chunks, page photographs) directly to Cloudinary without the backend ever buffering file bytes, keeping the API server's memory/bandwidth profile flat regardless of file size.	The backend never sees the file content itself before it lands in storage, so any content validation has to happen either client-side or after the fact.
Redux Toolkit primarily as RTK Query's cache host	A separate general-purpose Redux slice per feature; a non-Redux data-fetching library (e.g. React Query)	RTK Query's cache invalidation model (tag-based) fits the app's read-heavy, role-scoped data well, and reusing the same store avoids introducing a second state-management concept — but the project never actually needed cross-cutting client-only state, so a second general-purpose slice was never built (the one scaffolded <code>authSlice</code> was abandoned in favor of <code>localStorage</code>).	Authenticated-user state (token, role) lives outside Redux entirely, so it isn't reactive to Redux subscribers — components read it directly from storage where needed.
Time-driven exam/room state machines over user-triggered transitions	Requiring an explicit admin action to mark an exam "Ongoing" or "Completed"	An exam's actual availability window is defined by its schedule, not by whether an admin remembered to click a button — a scheduled sweep advancing status automatically keeps the system's state consistent with real time even if nobody is looking at the admin panel.	Introduces the class of "client displayed a status that has since changed" bugs documented in Section 25.2 — every screen that shows exam/room status needs to either poll or tolerate staleness deliberately.

27. Testing Strategy

27.1 Current State

CONFIRMED, NOT OVERSTATED

Jest, ts-jest, and Supertest are installed and fully configured in the backend (`test` , `test:e2e` , `test:cov` scripts all work), but the only test files present are NestJS's own generated boilerplate — `app.controller.spec.ts` (asserts `getHello()` returns a static string) and `test/app.e2e-spec.ts` (asserts `GET /` returns 200). No project-specific unit or integration test exists for any controller, service, or repository. The frontend has no testing dependencies installed at all — no Vitest, Jest, React Testing Library, Cypress, or Playwright.

27.2 Manual Testing Performed

Given the absence of automated coverage, verification during development relied on: TypeScript compilation (`tsc -b`) as a correctness gate on both repositories, manual end-to-end click-through of each feature across all three roles, and targeted database-level verification scripts for scheduling/timezone behavior. This substitutes for, but does not replace, real automated test coverage.

27.3 Testing Matrix — Priority Areas for Future Coverage

WORKFLOW	RISK IF UNTESTED	SUGGESTED TEST TYPE
Exam eligibility matching (Section 16.5)	A student sees/joins an exam they shouldn't, or misses one they should.	Unit tests on the matching query construction.
Room formation partitioning & pooling	Students dropped, duplicated, or assigned to no room.	Unit tests on <code>formExamRoomsAPI</code> with varied student/faculty counts.
Exam lifecycle sweep	An exam never advances, or advances early.	Unit tests with mocked time.
Role-based access on every endpoint	Privilege escalation.	E2E tests asserting a 403/404 for every wrong-role request.
Attempt scoring	Incorrect marks awarded.	Unit tests covering MCQ/MSQ scoring edge cases (partial MSQ selection, no answer, shuffled options).

28. Performance Considerations

28.1 Implemented

OPTIMIZATION	WHERE
Pagination	Every admin listing endpoint (students, faculty, exams, exam rooms) accepts page/limit query parameters rather than returning full collections.
Batch-loading to avoid N+1 queries	Exam room/attempt/answer listing endpoints resolve student and faculty display details via a single batch lookup (<code>findByIdsPreserveOrder</code>) keyed off the distinct ids in the result set, not one query per row.
Direct-to-storage uploads	Cloudinary signed uploads (Section 21) keep large file transfer off the API server entirely.
Database indexing	Every collection with a meaningful query pattern carries a purpose-built index (Section 12.2) — including the deliberate multikey-index split described in Section 25.1.
RTK Query caching	Tag-based cache invalidation avoids redundant refetches of unchanged data on the client.
Recording chunking	Exam session recordings upload in sequential chunks during the exam rather than one large file at the end, bounding memory use and giving partial-progress durability if the browser tab closes mid-exam.

28.2 Not Implemented (Future Opportunities)

- No response caching layer (Redis is a dead dependency, not an active cache — Section 20.4).
- No API-level rate limiting to protect against burst load or abuse (Section 19.2).
- No frontend code-splitting — the production build is a single JavaScript bundle (confirmed via a production build producing one ~3MB bundle with Vite's own chunk-size warning); route-based `React.lazy()` splitting would reduce initial load size.
- No compression middleware on the backend response pipeline.

29. Error Handling & Validation

29.1 Backend

NestJS's standard HTTP exception classes carry business-rule failures (Section 9.3); the global `ValidationPipe` catches malformed/incomplete request payloads before a controller method ever runs. Repository-layer failures are normalized to `InternalServerErrorException` so a raw database driver error never reaches the client. See Section 10.4 for the exact error response shape and Section 10.5 for the status codes in use.

29.2 Frontend

Every mutation call follows the same pattern: a try/catch around the RTK Query call, surfacing `error.data?.message` (the backend's own message) via a toast notification on failure, with a generic fallback string if the response doesn't carry one. Formik + Yup handle field-level validation before a request is even sent,

so most validation errors are caught client-side first — the backend's own validation is the authoritative, non-bypassable layer, not the only one.

29.3 Real-Time / Media Failures

LiveKit connection failures set a visible "Offline"/"Failed" status badge in both the student and faculty UI rather than silently failing; a LiveKit disconnect specifically is handled by the grace-period sweep (Section 20.2), not treated as an unrecoverable error.

30. Feature Traceability Matrix

Connects each major feature to every layer that implements it, so a change in one place can be traced to everything else it touches.

FEATURE	FRONTEND	API	SERVICE	DATABASE	EXTERNAL SERVICE
Login	LoginPage	POST /auth/login	AuthService	User, AuthActivityLog	—
Create Exam	CreateExamPage	POST /admin/exams	ExamManagement Service	Exam	—
Form Rooms	ExamDetailPage	POST /admin/exams/:id/rooms	ExamManagement Service	ExamRoom, ExamRoomAssignment	—
Join Lobby	LobbyPage	POST /student/exam-rooms/:examId/join	ExamAttemptService	ExamRoomAssignment	—
Admit Student	ProctoringDashboardPage	POST .../assignments/:id/admit	FacultyService	ExamRoomAssignment, ExamAttempt	—
Live Proctoring Video	ProctoringDashboardPage, ProctoredExamRoomPage	POST .../livekit-token	FacultyService / ExamAttemptService	ExamRoom	LiveKit
Written Answer Capture	WrittenAnswerCapture, MobileWrittenAnswerPage	/public/written-answer/*	PublicWrittenAnswerService	ExamAnswer	Cloudinary
Submit Exam	ExamRoomPage / ProctoredExamRoomPage	POST .../submit	ExamAttemptService	ExamAttempt, ExamAnswer	—
Evaluate Answer	EvaluationQueuePage	PATCH /faculty/evaluation/answers/:id	FacultyService	ExamAnswer	—
Publish Results	ExamDetailPage	POST .../publish-results	ExamManagement Service	Exam	—
Recording Upload	useExamRecorder hook	/student/exams/attempt/:id/recordings/*	ExamAttemptService	ExamRecording	Cloudinary
Password Reset	ForgotPasswordPage / ResetPasswordPage	/auth/forgot-password, /auth/reset-password	AuthService	User	— (console-logged, no email sent)

31. Future Improvements

Ordered roughly by how directly each addresses a gap already identified elsewhere in this document, not by difficulty.

IMPROVEMENT	ADDRESSES
Integrate an email/SMS provider	Section 2.2 — password reset currently only console-logs its link; no exam/result notifications exist at all.
Automated test coverage (unit + e2e)	Section 27 — currently only default framework boilerplate.
CI/CD pipeline for both repositories	Section 22 — builds/deployments are currently manual.
API rate limiting, especially on <code>/auth/*</code>	Section 19.2.
Remove or actually integrate the Redis dependency	Section 20.4 — currently a dead dependency with no consumer.
Fix the known schema inconsistencies	Section 12.1 — the misspelled <code>exam_room_assignment</code> collection name and the dead <code>admissionNumber</code> index, both low-risk but worth cleaning up in a deliberate migration.
Re-enable or remove the dormant profile-photo upload path	Section 21.2 — currently half-built (schema field + service method exist, controller wiring is commented out).
Frontend route-based code splitting	Section 28.2 — reduce the single large production bundle.
An admin-facing action audit log beyond login/logout	Section 25.2 — exam edits, evaluations, and student removals currently have no dedicated change history.
Automated or AI-assisted first-pass scoring for subjective answers, with faculty override	Section 2.3 — currently 100% manual.

32. Conclusion

XaminalityIQ is a coherent, purpose-built examination platform rather than a generic CRUD application with exams bolted on: its data model, service layer, and real-time infrastructure are all organized around the actual shape of running an exam — academic eligibility, a time-driven exam lifecycle, faculty-gated live supervision, and a clean separation between objective and subjective scoring. The system's two most architecturally significant properties are (1) authorization and eligibility computed at request time from the academic hierarchy rather than duplicated/cached, and (2) exam and room state that advances on real time via a scheduled sweep rather than requiring a human to remember to click a button — a choice that paid off in correctness but, as documented in

Section 25, also introduced a specific, real class of "client displayed a status that has since changed" defects that had to be found and fixed during development.

The system's honest gaps — no automated test coverage, no CI/CD, no rate limiting, no email delivery — are documented plainly in Sections 2.2, 22.2, and 33 rather than glossed over, since an accurate account of what is *not* yet built is as useful to a future engineer as an account of what is.

33. Glossary

TERM	MEANING
RBAC	Role-Based Access Control — authorization keyed on a fixed set of roles rather than per-user permissions.
JWT	JSON Web Token — the signed token format used for access, refresh, password-reset, and QR-upload authentication.
DTO	Data Transfer Object — a <code>class-validator</code> -decorated class defining and validating one API request or response shape.
SFU	Selective Forwarding Unit — the WebRTC server architecture LiveKit uses to route media between participants without a full mesh of peer connections.
ODM	Object-Document Mapper — Mongoose, mapping TypeScript classes to MongoDB documents.
AUTO exam	A self-paced exam mode with no live proctoring or room assignment.
PROCTORING exam	An invigilated exam mode requiring room formation and live faculty supervision.
Exam Section	A named question-grouping within one exam (e.g. "Section A") — not the academic <code>Section</code> (class/division) entity.
Room Formation	The admin action that partitions matched students into proctoring rooms and assigns invigilating faculty.
Waiting Room / Lobby	The pre-admission state a student sits in for a PROCTORING exam before their assigned faculty admits them.
Attempt	One student's specific run at one exam (<code>ExamAttempt</code>) — distinct from the <code>Exam</code> definition itself.
Sweep	A scheduled background job that advances state based on wall-clock time rather than a user action (Section 24.3).
Self-serve profile completion	The pattern where a student/faculty account is created with only minimal identity/placement fields, and completes the rest of their own profile after first login.

34. Appendix

A.1 Enum Reference (`src/utils/enum.ts`)

ENUM	VALUES
UserRole	ADMIN, FACULTY, STUDENT
ExamMode	AUTO, PROCTORING
ExamStatus	DRAFT, PUBLISHED, ONGOING, COMPLETED, RESULTS_PUBLISHED
QuestionType	MCQ, MSQ, WRITTEN, TYPING
AttemptStatus	NOT_STARTED, IN_PROGRESS, SUBMITTED, COMPLETED
SubmissionTrigger	MANUAL, TIMER_EXPIRY, INTEGRITY_AUTO_SUBMIT, FACULTY_REMOVED, CONNECTION_LOSS
RoomAssignmentStatus	WAITING, ADMITTED, IN_PROGRESS, REJECTED, REMOVED, DISCONNECTED, COMPLETED
ExamRoomStatus	SCHEDULED, ACTIVE, CLOSED (effectively unused in practice — real-time status is derived from schedule, not this field; see prior engineering note in the codebase)
ChatSenderRole	FACULTY, STUDENT
ChatRecipientType	INDIVIDUAL, BROADCAST_ROOM, BROADCAST_EXAM
MediaStatus	PENDING_UPLOAD, UPLOAD_COMPLETE (exact member set per usage context — recording and QR-answer flows share the concept of a pending vs. complete upload)
ViolationType	Tab-switch and fullscreen-exit style integrity events (Section 20.3)
StudentStatus / FacultyStatus	ACTIVE and related lifecycle states used to filter eligible students/faculty (e.g. room formation only considers ACTIVE faculty)

A.2 Complete Collection List (31)

Full field-level detail for every collection below is in Section 12.2.

batches · courses · batchcourses · departments · batchdepartments · sections · subjects · users · authactivitylogs · faculties · facultypersonaldetails · facultycontactinformations · facultyaddresses · facultyemploymentdetails · facultyeducationhistories · facultyworkexperiences · students · studentacademicdetails · studentpersonaldetails · studentcontactinformations · studentaddressdetails · studenteducationhistories · studentparentdetails · exams · exam_questions · exam_attempts · exam_answers · exam_recordings · exam_rooms · exam_room_assignment · exam_room_chat_messages

A.3 Sample Success/Error Payloads

See Section 10.3–10.4 for the standard envelope shapes and Section 10.2 for full worked examples on representative endpoints (login, create exam, form rooms, join lobby, admit student, save answer, submit exam).

A.4 HTTP Status Code Mapping

See Section 10.5.

A.5 Repository Locations

REPOSITORY	PURPOSE
XaminiyIQ-Server	NestJS backend — github.com/sanjaikannang/XaminiyIQ-Server
XaminiyIQ-Client	React frontend — github.com/sanjaikannang/XaminiyIQ-Client